GROUP WORK PROJECT # 2 MScFE 600: FINANCIAL DATA

GROUP NUMBER: 8019

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Kolapo Adedipe	Nigeria	adcolapsmm9@yahoo.com	
Loice Wanjiru Nguku	Kenya	loicenguku@outlook.com loicenguku21@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).	
Team member 1	**Kolapo Adedipe**
Team member 2	**Loice Wanjiru Nguku**
Team member 3	

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed. **Note:** You may be required to provide proof of your outreach to non-contributing members upon request.

1. On top left of your screen click on File →Download → Microsoft Word (.docx) to download this template
2. **Upload the template in Google Drive and share it with your group members**
3. **Delete this page with the requirements before submitting your report. Leaving them will result in an increased similarity score on Turnitin.**

Keep in mind the following:

- Make sure you address all the questions in the GWP assignment document published in the Course Overview.
- Follow the "Submission requirements and format' instructions included in each Group Work Project Assignment, including report length.
- **Including in-text citations and related references is mandatory for all submissions.** You will receive a '0' grade for missing in-text citations and references, or penalties for partial completion. Use the **In-Text Citations and References Guide** to learn how to include them.
- Additional writing aids: Anti-Plagiarism Guide, Academic Writing Guide, Online Writing Resources.
- To avoid an increase in the Turnitin similarity score, **DO NOT copy the questions** from the GWP assignment document.
- Submission format tips:
  - Use the same font type and size and same format throughout your report. You can use Calibri 11, Arial 10, or Times 11.
  - Do NOT split charts, graphs, and tables between two separate pages.
  - Always include the axes labels and scales in your graphs as well as an explanation of how the data should be read.
- Use the LIRN Library for your research. It can be accessed via the left navigation pane inside the WQU learning platform.
- Carefully read **Academic Policy on the use of AI** explaining how the use of AI tools is restricted and regulated. Severe penalties apply for excessive and improper use of AI

The PDF file with your report must be uploaded separately from the zipped folder that includes any other types of files. This allows Turnitin to generate a similarity report.

PART 1

Q1(i)

The paper uses **historical stock market data** to predict stock market movements, specifically focusing on **Exchange-Traded Funds (ETFs)** in emerging markets. The datasets include the following key attributes:

- **Opening price (Open)**
- **Highest price (High)**
- **Lowest price (Low)**
- **Closing price (Close)**
- **Adjusted close price (Adjusted Close)**
- **Trading volume (Volume)**
- **Dividends yield and capital gain distributions**

These datasets were obtained from **Yahoo Finance** and span a time frame from **December 12, 2009, to January 1, 2020**. The focus is on three ETFs:

1. **iShares MSCI Chile ETF (ECH)**
2. **iShares MSCI Brazil ETF (EWZ)**
3. **iShares Core S&P 500 ETF (IVV)**

Derivation of Technical Indicators:

The paper employs the **Pandas Technical Analysis Library (Pandas TA)** to compute **216 technical indicators** from the six fundamental attributes (Open, High, Low, Close, Adjusted Close, and Volume). These indicators are categorized into the following groups:

- **Candles**
- **Cycles**
- **Momentum**
- **Overlap**
- **Performance**
- **Statistics**
- **Trend**
- **Utility**
- **Volatility**
- **Volume**

Examples of technical indicators derived from the data include:

- **Moving Averages (SMA, EMA)**
- **Relative Strength Index (RSI)**
- **Bollinger Bands (BBP)**
- **Stochastic Oscillator (STOCH)**
- **Williams %R (WILLR)**
- **Correlation Trend Indicator (CTI)**
- **On Balance Volume (OBV)**

- **Price Volume Rank (PVR)**

These indicators are calculated using mathematical formulas applied to the historical price and volume data. For instance:

- **Stochastic RSI**: Combines stochastic oscillator and RSI to measure momentum.
- **Z-score**: Normalizes price data using the mean and standard deviation of closing prices.

Data Preprocessing:

Before deriving the technical indicators, the data undergoes preprocessing steps:

1. **Class Assignment**: A binary classification strategy is used to predict whether the ETF's opening price will increase or decrease.
2. **Data Normalization**: A min-max normalization approach is applied to scale the data between 0 and 1.
3. **Data Cleaning**: Missing or incomplete data (e.g., due to the calculation of indicators like SMA) are removed to ensure data quality.

In summary, the paper uses **historical stock market data** to derive **technical indicators** through mathematical transformations, which are then used as input features for predicting stock market movements. The focus is on reducing computational complexity by selecting the most relevant indicators while maintaining or improving prediction accuracy (Deville, 2008; Antoniewicz and Heinrichs, 2014).

Q1(ii)

Importance of Using Technical Indicators in Forecasting Stock Price Trends

Technical indicators play a crucial role in forecasting stock price trends, especially in the context of emerging markets where volatility and unpredictability are often higher. Here are the key reasons why these indicators are important:

1. Capturing Market Trends and Patterns

- **Trend Identification**: Indicators like Moving Averages (MA), Bollinger Bands, and Correlation Trend Indicators (CTI) help identify the direction of market trends (upward, downward, or sideways). This is essential for making informed buy/sell decisions.
- **Cyclical Behavior**: Indicators such as Even Better SineWave (EBSW) and Stochastic RSI are designed to detect cyclical patterns in stock prices, which are particularly useful in markets with recurring trends.

2. Measuring Market Momentum

- **Momentum Indicators**: Tools like the Relative Strength Index (RSI) and Stochastic Oscillator measure the speed and strength of price movements. They help traders identify overbought or oversold conditions, signaling potential reversals.
- **Volume-Based Indicators**: On Balance Volume (OBV) and Price Volume Rank (PVR) incorporate trading volume to confirm the strength of a trend. High volume during price increases

suggests strong buyer interest, while high volume during price decreases indicates strong selling pressure.

3. Reducing Noise and Enhancing Signal Clarity

- **Filtering Noise**: Indicators like the Even Better SineWave and Z-score help filter out market noise, making it easier to focus on significant price movements.
- **Smoothing Data**: Moving Averages smooth out short-term fluctuations, providing a clearer view of the underlying trend.

4. Risk Management

- **Volatility Indicators**: Bollinger Bands and Average True Range (ATR) measure market volatility, helping traders set stop-loss and take-profit levels. This is crucial for managing risk in highly volatile emerging markets.
- **Support and Resistance Levels**: Indicators like Williams %R and Stochastic RSI help identify key support and resistance levels, which are critical for setting entry and exit points.

5. Enhancing Predictive Models

- **Feature Selection**: The paper emphasizes the importance of selecting the most relevant technical indicators to improve the accuracy of predictive models. By reducing the dimensionality of the dataset, the model becomes computationally efficient and less prone to overfitting.
- **Nonlinear Relationships**: Neural networks and other machine learning models can capture complex, nonlinear relationships between indicators and price movements, which traditional statistical methods might miss.

6. Adaptability to Market Conditions

- **Dynamic Adjustment**: Many technical indicators can be adjusted for different time frames and market conditions, making them versatile tools for both short-term traders and long-term investors.
- **Emerging Markets**: In emerging markets, where economic conditions can change rapidly, technical indicators provide a flexible framework for adapting to new information and market dynamics.

7. Empirical Validation

- **Historical Performance**: The use of technical indicators is backed by extensive empirical research and historical performance data. For example, the paper demonstrates that selecting the top 5% of technical indicators can improve prediction accuracy by approximately 2% while reducing computational costs by 84.68%.

8. Behavioral Insights

- **Market Psychology**: Indicators like the Balance of Power (BOP) and On Balance Volume (OBV) provide insights into market psychology, helping traders understand whether the market is driven by accumulation (buying) or distribution (selling).

Conclusion

Technical indicators are indispensable tools for forecasting stock price trends, particularly in the volatile and unpredictable environment of emerging markets. They provide a structured way to analyze market data, identify trends, measure momentum, and manage risk. By integrating these indicators into predictive models, as demonstrated in the paper, investors and traders can enhance their decision-making processes, improve prediction accuracy, and achieve better investment outcomes. The systematic selection and optimization of these indicators further ensure that the models remain computationally efficient and adaptable to changing market conditions (Miralles-Quiros et al., 2019).

Q2(i)

Fund Description: iShares MSCI Chile ETF (ECH)

Overview

The **iShares MSCI Chile ETF (ECH)** is an exchange-traded fund designed to track the investment results of the **MSCI Chile IMI 25/50 Index**. This index measures the performance of large, mid, and small-cap equities in the Chilean market. ECH provides investors with exposure to a broad range of Chilean companies, offering a way to invest in one of Latin America's most dynamic emerging markets (Hegde and McDermott, 2004).

Asset Type

ECH primarily invests in **equities** listed on the Chilean stock exchange. The fund holds a diversified portfolio of stocks across various sectors, with a focus on companies that are representative of the Chilean economy. As of the latest data, the fund's top sectors include:

- **Financials (21.53%)**
- **Materials (21.28%)**
- **Utilities (18.73%)**
- **Consumer Staples (13.92%)**
- **Energy (8.34%)**

These sectors collectively account for approximately **83.8%** of the fund's total holdings, reflecting the economic composition of Chile, which is heavily influenced by natural resources, financial services, and utilities (Chen and Chen, 2015; Jiang et al., 2020; Zhang et al., 2021).

Price History

The price history of ECH reflects the volatility and growth potential typical of emerging markets. Over the analyzed period from **December 12, 2009, to January 1, 2020**, the fund's price exhibited significant fluctuations, influenced by both local and global economic factors. Key price statistics include:

- **Minimum Price**: $29.30
- **Maximum Price**: $80.25
- **Median Price**: $46.48
- **Mean Price**: $50.10

The fund's price history shows periods of rapid growth, particularly during commodity booms, as well as sharp declines during economic downturns or political instability in Chile (Ananthi and Vijayakumar, 2021; Cagliero et al., 2020).

Performance and Volatility

ECH's performance is closely tied to the Chilean economy, which is heavily reliant on copper exports and other natural resources. The fund's price movements are often correlated with global commodity prices and economic conditions in major trading partners like China and the United States. Key performance metrics include:

- **Annualized Volatility**: Higher than developed market ETFs, reflecting the inherent risks of emerging markets.
- **Dividend Yield**: ECH offers a competitive dividend yield, making it attractive for income-focused investors.

Market Exposure

ECH provides exposure to a mix of large-cap and mid-cap Chilean companies, offering a balanced representation of the country's equity market. The fund's top holdings typically include major Chilean corporations such as:

- **Banco de Chile**
- **Enel Americas**
- **Sociedad Química y Minera de Chile (SQM)**
- **Empresas CMPC**

These companies are leaders in their respective sectors and play a significant role in the Chilean economy (Tang et al., 2019).

Conclusion

The **iShares MSCI Chile ETF (ECH)** is a valuable tool for investors seeking exposure to the Chilean equity market. Its diversified portfolio across key sectors, combined with the growth potential of an emerging market, makes it an attractive option for both growth and income investors. However, the fund's price history and performance highlight the importance of understanding the risks associated with emerging markets, including economic volatility and political instability. For investors willing to navigate these risks, ECH offers a compelling opportunity to participate in the growth of one of Latin America's most promising economies (De Haan et al., 2016).

Q2(ii)

Why the Authors Chose a Classification Problem Over Regression

The authors decided to run a **classification problem** rather than a regression problem for the following reasons:

1. **Focus on Trend Direction**:

- The primary goal of the study is to predict the **direction of stock price movements** (up or down) rather than the exact price values. Classification is well-suited for this purpose as it categorizes outcomes into discrete classes (e.g., increase or decrease). This aligns with the practical needs of investors, who are often more interested in whether the market will rise or fall rather than precise price predictions (Liang et al., 2017).

2. **Simplification of the Problem**:
   - Stock price movements are inherently noisy and influenced by numerous unpredictable factors. By framing the problem as a classification task, the authors simplify the prediction task to a binary outcome (increase or decrease), which is easier to model and interpret compared to regression, where the model must predict continuous values (Zhao et al., 2021).
3. **Interpretability**:
   - Classification results are more interpretable for decision-making. For example, a model predicting whether the price will go up or down provides clear actionable insights, whereas regression outputs (e.g., predicted price values) require additional interpretation to determine whether the predicted change is significant (Barra et al., 2020).
4. **Computational Efficiency**:
   - Classification models, especially when combined with feature selection, can be computationally more efficient than regression models, particularly when dealing with high-dimensional data like the 216 technical indicators used in this study (Chen et al., 2019; Nelson et al., 2017).

Q3(i)

Section 2: Data

The **Data** section focuses on the datasets used in the study, their sources, and the preprocessing steps applied to prepare the data for analysis. This section can be divided into the following subcategories:

2.1 Data Sources

- **Description**: The datasets used in the study are obtained from **Yahoo Finance**, a widely used platform for financial data. The study focuses on three ETFs:
  - **iShares MSCI Chile ETF (ECH)**
  - **iShares MSCI Brazil ETF (EWZ)**
  - **iShares Core S&P 500 ETF (IVV)**
- **Time Frame**: The data spans from **December 12, 2009, to January 1, 2020**, excluding the pandemic period to avoid atypical market behavior.
- **Attributes**: The datasets include the following key attributes for each ETF:
  - **Open**: Opening price
  - **High**: Highest price of the day
  - **Low**: Lowest price of the day
  - **Close**: Closing price

- **Adjusted Close**: Adjusted closing price accounting for splits and dividends
- **Volume**: Trading volume

2.2 Data Processing

- **Technical Indicators**: The study uses the **Pandas Technical Analysis Library (Pandas TA)** to compute **216 technical indicators** from the six fundamental attributes (Open, High, Low, Close, Adjusted Close, and Volume). These indicators are categorized into groups such as Momentum, Volatility, Trend, and Volume.
- **Class Assignment**: A binary classification variable $\Gamma(t)$ is defined to predict the direction of the opening price movement.
- **Normalization**: The data is normalized using a **min-max scaling** approach to ensure all features are on a comparable scale.
- **Data Cleaning**: Missing or incomplete data (e.g., due to the calculation of indicators like SMA) are removed to ensure data quality. The final dataset spans from **January 1, 2009, to January 1, 2020**.

2.3 Feature Engineering

- **Feature Selection**: The study employs various statistical measures to identify the most salient features (technical indicators) for predicting ETF trends. These measures include:
  - **Low Variance**: Removes features with negligible variance.
  - **Chi-Squared Test**: Evaluates the independence of features from the target variable.
  - **LASSO Regression**: Applies regularization to select relevant features.
  - **Tree-Based Feature Selection**: Uses ensemble methods to rank feature importance.
  - **Pearson's Correlation**: Identifies linear relationships between features and the target.
  - **Principal Feature Analysis**: Retains features that capture maximum variability.
  - **Mean Absolute Difference (MAD)**: Ranks features based on their discriminative power.
  - **Dispersion Ratio (DR)**: Measures feature relevance based on dispersion.
- **Subset Selection**: The top 5% of features (e.g., 10 out of 216) are selected for each ETF to reduce computational complexity while maintaining prediction accuracy (O'Hara et al., 2000).

2.4 Data Splitting

- **Cross-Validation**: The dataset is partitioned into **10 folds** for cross-validation. This ensures robust evaluation of the model's performance by training and testing on different subsets of the data (Fang et al., 2024; Matuozzo et al., 2023; Verma et al., 2023a,b,c).
- **Training and Testing**: For each fold, one partition is used for testing, and the remaining nine are used for training. This process is repeated for all partitions, and the results are averaged to obtain a reliable estimate of model accuracy.

2.5 Data Summary

- **Descriptive Statistics**: Key statistics for the opening price of each ETF are provided, including minimum, maximum, median, mean, and quartile values.
- **Cumulative Movement**: The cumulative movement of the classification variable $\Gamma(t)$ is visualized to show the overall trend direction for each ETF over the study period.

Conclusion

The **Data** section provides a comprehensive overview of the datasets, preprocessing steps, and feature engineering techniques used in the study. By carefully selecting and processing the data, the authors ensure that the predictive models are built on a solid foundation, enabling accurate and efficient trend prediction for the selected ETFs. The subcategories—**Data Sources**, **Data Processing**, **Feature Engineering**, **Data Splitting**, and **Data Summary**—organize the information logically, making it easier to understand the methodology and replicate the study (Almehmadi, 2021; Verma et al., 2022).

Q3(ii)

The **Methodology** section describes the techniques and algorithms used to analyze the data and build predictive models. This section can be divided into the following subcategories:

3.1 Feature Selection Techniques

The study employs a variety of statistical and machine learning methods to identify the most relevant features (technical indicators) for predicting ETF trends (Shearer, 2000). These techniques include:

1. **Low Variance**:
   - Removes features with negligible variance, as they provide little information for prediction.
2. **Chi-Squared Test**:
   - Evaluates the independence of features from the target variable.
3. **LASSO (Least Absolute Shrinkage and Selection Operator)**:
   - A regularization technique that penalizes the absolute size of feature coefficients, effectively performing feature selection.
4. **Tree-Based Feature Selection**:
   - Uses ensemble methods like Extra Trees Classifier to rank feature importance based on their contribution to model performance.
5. **Pearson's Correlation**:
   - Measures the linear relationship between features and the target variable.
6. **Principal Feature Analysis**:
   - Retains features that capture maximum variability in the data using principal component analysis (PCA) and k-means clustering.

7. **Mean Absolute Difference (MAD)**:
    - Ranks features based on their discriminative power.
8. **Dispersion Ratio (DR)**:
    - Measures feature relevance based on the ratio of arithmetic mean to geometric mean.

3.2 Model Architecture

- **Multilayer Perceptron (MLP)**:
    - The study uses an MLP, a type of neural network, for binary classification (predicting the direction of price movements).
    - The MLP consists of:
        - **Input Layer**: Number of nodes equals the number of input features (e.g., 216 if all features are used).
        - **Hidden Layer**: Number of nodes equals input features plus classes divided by 2 (e.g., 109 nodes).
        - **Output Layer**: Single node for binary classification.
    - Activation Function: Logistic (sigmoid) for binary classification.
    - Solver: Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) for optimization.
    - Training: Backpropagation with adaptive learning rate and early stopping to prevent overfitting.

3.3 Cross-Validation

- **K-Fold Cross-Validation**:
    - The dataset is split into **10 folds** to evaluate model performance robustly.
    - For each fold, the model is trained on 9 partitions and tested on the remaining partition.
    - The process is repeated for all folds, and the results are averaged to obtain a reliable estimate of accuracy.
    - This approach ensures that the model generalizes well to unseen data.

3.4 Evaluation Metrics

- **Accuracy**:
    - The primary metric for evaluating model performance, defined as the percentage of correctly predicted price movements (up or down).
- **Computational Efficiency**:
    - The study emphasizes reducing computational costs by selecting only the top 5% of features, which improves training time and resource usage.
- **Percentage Gain**:
    - The improvement in accuracy achieved by using feature selection compared to using the full set of features.

3.5 Experimental Setup

- **Feature Subsets**:
  - The study evaluates the performance of different subsets of features (e.g., top 5%, top 10%) to determine the optimal balance between accuracy and computational efficiency.
- **Early Stopping**:
  - Implemented during MLP training to prevent overfitting and optimize computational resources.
- **Comparison Across ETFs**:
  - The methodology is applied to three ETFs (ECH, EWZ, and IVV) to compare performance across emerging and developed markets.

3.6 Algorithm Summary

- **Algorithm 1 (Cross-Validation)**:
  - Describes the steps for performing k-fold cross-validation, including dataset partitioning, model training, and accuracy calculation.
- **Algorithm 2 (Feature Selection and Evaluation)**:
  - Outlines the process for calculating technical indicators, selecting salient features, and evaluating model performance using the MLP.

Conclusion

The **Methodology** section provides a detailed description of the techniques and algorithms used in the study. By employing a combination of feature selection methods, neural network architecture, and robust evaluation techniques, the authors ensure that their predictive models are both accurate and computationally efficient. The subcategories—**Feature Selection Techniques**, **Model Architecture**, **Cross-Validation**, **Evaluation Metrics**, **Experimental Setup**, and **Algorithm Summary**—organize the methodology logically, making it easier to understand and replicate the study (Jamshed et al., 2019; Sun et al., 2017).

Q3(iii)

Descriptive statistics and models serve different purposes in data analysis, and they can be clearly divided based on their roles and applications (Ecer et al., 2020). Here's how they can be separated:

Descriptive Statistics

Descriptive statistics summarize and describe the main features of a dataset. They are used to understand the data's structure, distribution, and relationships without making predictions or inferences. Examples include:

1. **Measures of Central Tendency**:
    - Mean, median, and mode.
    - Example: The mean opening price of ECH is $50.10.
2. **Measures of Dispersion**:
    - Variance, standard deviation, range, and interquartile range (IQR).
    - Example: The standard deviation of ECH's opening price indicates its volatility.
3. **Correlation Analysis**:
    - Measures the strength and direction of the linear relationship between two variables.
    - **Pearson's Correlation**.
        - Example: Calculating the correlation between the opening price and trading volume to understand their relationship.
4. **Distribution Analysis**:
    - Histograms, box plots, and skewness/kurtosis measures.
    - Example: Visualizing the distribution of ECH's opening price over time.
5. **Feature Ranking**:
    - Techniques like Mean Absolute Difference (MAD) and Dispersion Ratio (DR) rank features based on their discriminative power or variability.
    - Example: Ranking technical indicators by their MAD values to identify the most relevant features.

Models

Models are used to make predictions or inferences based on the data. They involve building mathematical or computational structures that capture relationships between input features and target variables (Reddy et al., 2020). Examples include:

1. **Predictive Models**:
    - **LASSO (Least Absolute Shrinkage and Selection Operator)**:
        - A regression model that performs feature selection by penalizing the absolute size of coefficients.
        - Example: Using LASSO to select the most relevant technical indicators for predicting ETF price movements.
    - **Multilayer Perceptron (MLP)**:
        - A neural network model used for classification or regression tasks.
        - Example: Training an MLP to classify whether the opening price of ECH will increase or decrease based on selected technical indicators.
2. **Ensemble Models**:
    - **Tree-Based Models (e.g., Extra Trees Classifier)**:
        - Ensemble methods that combine multiple decision trees to improve prediction accuracy and robustness.

        - Example: Using Extra Trees Classifier to rank feature importance for ETF trend prediction.
3. **Evaluation Metrics**:
    - Metrics like accuracy, precision, recall, and F1-score are used to evaluate model performance.
    - Example: Calculating the accuracy of the MLP model in predicting the direction of ECH's price movements.
4. **Cross-Validation**:
    - A technique for assessing how well a model generalizes to unseen data by partitioning the dataset into training and testing subsets.
    - Example: Using 10-fold cross-validation to evaluate the MLP's performance.

Key Differences

Aspect	**Descriptive Statistics**	**Models**
Purpose	Summarize and describe data	Make predictions or inferences
Examples	Mean, variance, Pearson's correlation, MAD, DR	LASSO, MLP, Extra Trees Classifier
Output	Insights into data structure and relationships	Predictions (e.g., class labels, continuous values)
Dependency on Data	Standalone; no training required	Requires training on data
Complexity	Simple calculations and visualizations	Complex mathematical or computational structures

---

Conclusion

Descriptive statistics and models are distinct but complementary components of data analysis. Descriptive statistics provide a foundational understanding of the data, while models leverage this understanding to make predictions or inferences By clearly separating these two aspects, researchers can ensure a logical and structured approach to data analysis, as demonstrated in the study. For example:

- **Descriptive Statistics**: Pearson's correlation helps identify relationships between features.
- **Models**: LASSO and MLP use these relationships to build predictive models for ETF trend forecasting.

Q3(iv)

The **Methodology** section outlines the techniques and processes used to analyze the data and build predictive models (Lu et al., 2007). It is divided into the following subcategories:

3.1 Feature Selection Techniques

The study employs a variety of statistical and machine learning methods to identify the most relevant features (technical indicators) for predicting ETF trends. These techniques include:

1. **Low Variance**: Removes features with negligible variance.
2. **Chi-Squared Test**: Evaluates the independence of features from the target variable.
3. **LASSO (Least Absolute Shrinkage and Selection Operator)**: A regularization technique that penalizes the absolute size of feature coefficients.
4. **Tree-Based Feature Selection**: Uses ensemble methods like Extra Trees Classifier to rank feature importance.
5. **Pearson's Correlation**: Measures the linear relationship between features and the target variable.
6. **Principal Feature Analysis**: Retains features that capture maximum variability using PCA and k-means clustering.
7. **Mean Absolute Difference (MAD)**: Ranks features based on their discriminative power.
8. **Dispersion Ratio (DR)**: Measures feature relevance based on the ratio of arithmetic mean to geometric mean.

3.2 Model Architecture

- **Multilayer Perceptron (MLP)**:
  - The study uses an MLP, a type of neural network, for binary classification (predicting the direction of price movements).
  - The MLP consists of:
    - **Input Layer**: Number of nodes equals the number of input features.
    - **Hidden Layer**: Number of nodes equals input features plus classes divided by 2
    - **Output Layer**: Single node for binary classification.
  - Activation Function: Logistic (sigmoid) for binary classification.
  - Solver: Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) for optimization.
  - Training: Backpropagation with adaptive learning rate and early stopping to prevent overfitting.

3.3 Cross-Validation

- **K-Fold Cross-Validation**:
  - The dataset is split into **10 folds** to evaluate model performance robustly.
  - For each fold, the model is trained on 9 partitions and tested on the remaining partition.
  - The process is repeated for all folds, and the results are averaged to obtain a reliable estimate of accuracy.

3.4 Evaluation Metrics

- **Accuracy**: The percentage of correctly predicted price movements (up or down).
- **Computational Efficiency**: The reduction in computational costs achieved by selecting only the top 5% of features.
- **Percentage Gain**: The improvement in accuracy achieved by using feature selection compared to using the full set of features.

3.5 Experimental Setup

- **Feature Subsets**: The study evaluates the performance of different subsets of features (e.g., top 5%, top 10%) to determine the optimal balance between accuracy and computational efficiency.
- **Early Stopping**: Implemented during MLP training to prevent overfitting and optimize computational resources.
- **Comparison Across ETFs**: The methodology is applied to three ETFs (ECH, EWZ, and IVV) to compare performance across emerging and developed markets.

3.6 Algorithm Summary

- **Algorithm 1 (Cross-Validation)**: Describes the steps for performing k-fold cross-validation.
- **Algorithm 2 (Feature Selection and Evaluation)**: Outlines the process for calculating technical indicators, selecting salient features, and evaluating model performance using the MLP.

Optimization Process of Technical Indicators

1. Feature Selection

The authors optimize technical indicators by systematically selecting the most relevant features using statistical and machine learning techniques. This process involves:

- **Ranking Features**: Techniques like MAD, DR, and Pearson's correlation rank features based on their discriminative power or relationship with the target variable.
- **Reducing Dimensionality**: By selecting only the top 5% of features, the authors reduce the input space, which improves computational efficiency and model performance.

2. Model Training

- **MLP Configuration**: The MLP is configured with an adaptive learning rate and early stopping to optimize training. This ensures that the model generalizes well to unseen data and avoids overfitting.
- **Cross-Validation**: The use of 10-fold cross-validation ensures that the model's performance is robust and reliable.

3. Performance Evaluation

- **Accuracy Improvement**: The authors demonstrate that using the optimized subset of features improves prediction accuracy by approximately 2% compared to using the full set of features.
- **Computational Efficiency**: The reduction in the number of features leads to an 84.68% reduction in training time, making the model more efficient.

Importance of Optimizing Technical Indicators

1. **Improved Predictive Power**:
    - By selecting the most relevant features, the model focuses on the indicators that have the strongest relationship with the target variable, leading to more accurate predictions.
2. **Computational Efficiency**:
    - Reducing the number of features decreases the computational complexity of the model, making it faster to train and more scalable for large datasets.
3. **Generalization**:
    - Optimizing features helps prevent overfitting, ensuring that the model performs well on unseen data.
4. **Interpretability**:
    - A smaller set of features makes it easier to interpret the model's predictions and understand the factors driving stock price movements.
5. **Resource Optimization**:
    - Efficient use of computational resources is crucial for real-time applications, such as algorithmic trading, where speed and accuracy are critical.

Conclusion

The optimization of technical indicators is a critical step in building effective predictive models for stock market trends. By systematically selecting the most relevant features and configuring the MLP for optimal performance, the authors achieve a balance between accuracy and computational efficiency. This approach not only improves the predictive power of the model but also ensures that it is practical and scalable for real-world applications.

Q4(i)

In the context of the paper, a **feature** refers to a specific measurable property or characteristic derived from the historical stock market data that is used as an input to the predictive model. Features are essential for training machine learning models, as they capture relevant information about the data that can help predict future trends.

Types of Features in the Paper

The paper considers **technical indicators** as features (Chen et al., 2021b). These indicators are mathematical calculations based on historical price, volume, or open interest data, and they are used to forecast future price movements. The authors compute **216 technical indicators** using the **Pandas Technical Analysis Library (Pandas TA)**. These indicators are derived from the following fundamental attributes of the ETFs:

- **Open**: Opening price
- **High**: Highest price of the day
- **Low**: Lowest price of the day
- **Close**: Closing price
- **Adjusted Close**: Adjusted closing price accounting for splits and dividends
- **Volume**: Trading volume

Categories of Features (Technical Indicators)

The 216 technical indicators are grouped into the following categories:

1. **Candles**: Patterns derived from price movements (e.g., Doji, Hammer).
2. **Cycles**: Indicators that identify cyclical patterns in price data (e.g., Even Better SineWave).
3. **Momentum**: Measures the speed and strength of price movements (e.g., Relative Strength Index (RSI), Stochastic Oscillator).
4. **Overlap**: Indicators that overlay on the price chart (e.g., Moving Averages, Bollinger Bands).
5. **Performance**: Metrics that evaluate the performance of the asset (e.g., Rate of Change).
6. **Statistics**: Statistical measures of price data (e.g., Z-score, Correlation Trend Indicator).
7. **Trend**: Indicators that identify the direction of the market trend (e.g., Moving Average Convergence Divergence (MACD)).
8. **Utility**: Helper functions or transformations (e.g., Normalization).
9. **Volatility**: Measures of price variability (e.g., Average True Range (ATR)).
10. **Volume**: Indicators based on trading volume (e.g., On Balance Volume (OBV), Price Volume Rank (PVR)).

Examples of Features

Here are some specific examples of features (technical indicators) used in the paper:

1. **Balance of Power (BOP)**:
    - Measures buying and selling pressure.
2. **Relative Strength Index (RSI)**:
    - Measures the magnitude of recent price changes to evaluate overbought or oversold conditions.
3. **Bollinger Bands Percent (BBP)**:
    - Quantifies the price relative to the upper and lower Bollinger Bands.
4. **On Balance Volume (OBV)**:
    - Tracks cumulative buying and selling pressure based on volume.
5. **Z-score**:
    - Normalizes price data using the mean and standard deviation.
    - Formula:
    - Z-score=Price−MeanStandard Deviation

Feature Selection and Optimization

The authors use various statistical and machine learning techniques to select the most relevant features (technical indicators) for predicting ETF trends. These techniques include:

- **Low Variance**: Removes features with negligible variance.
- **Chi-Squared Test**: Evaluates the independence of features from the target variable.
- **LASSO**: Performs feature selection by penalizing the absolute size of coefficients.
- **Tree-Based Feature Selection**: Ranks features based on their importance.
- **Pearson's Correlation**: Measures the linear relationship between features and the target variable.
- **Principal Feature Analysis**: Retains features that capture maximum variability.
- **Mean Absolute Difference (MAD)**: Ranks features based on their discriminative power.
- **Dispersion Ratio (DR)**: Measures feature relevance based on dispersion.

By selecting only the top 5% of features, the authors reduce the dimensionality of the dataset, improving computational efficiency and model performance.

Conclusion

In the paper, a **feature** is a technical indicator derived from historical stock market data, such as price and volume. These features are used as inputs to the predictive model to forecast future price movements. The authors optimize these features by selecting the most relevant ones using statistical and machine learning techniques, ensuring that the model is both accurate and computationally efficient. This approach highlights the importance of feature engineering and selection in building effective predictive models for financial markets (Johnson, 2021).

Q4(ii)

In data analysis and machine learning, the terms **feature**, **method**, and **model** refer to distinct concepts, each playing a unique role in the process of analyzing data and making predictions. Here's how they are distinguished:

---

1. Feature

- **Definition**: A feature is a measurable property or characteristic of the data that is used as an input to a model. It represents a specific aspect of the data that can help in making predictions or understanding patterns.
- **Role**: Features capture relevant information about the data and serve as the building blocks for models.
- **Examples**:
  - In stock market prediction, features include technical indicators like **Relative Strength Index (RSI)**, **Moving Averages (MA)**, and **On Balance Volume (OBV)**.
  - In a dataset of houses, features might include **square footage**, **number of bedrooms**, and **location**.

---

2. Method

- **Definition**: A method refers to a specific technique or algorithm used to process data, analyze relationships, or build models. Methods are the tools or procedures applied to features or data to achieve a particular goal.
- **Role**: Methods are used to transform, analyze, or model data. They can be statistical, mathematical, or computational in nature.
- **Examples**:
  - **Feature Selection Methods**: Techniques like **LASSO**, **Pearson's Correlation**, and **Principal Feature Analysis** are used to identify the most relevant features.
  - **Data Preprocessing Methods**: Techniques like normalization, scaling, and handling missing data.
  - **Statistical Methods**: Techniques like regression analysis, hypothesis testing, or clustering.

---

3. Model

- **Definition**: A model is a mathematical or computational representation of a system or process that is trained on data to make predictions or inferences. It is the end product that uses features and methods to generate outputs.
- **Role**: Models capture the relationships between input features and output targets, enabling predictions or decisions based on new data.
- **Examples**:
  - **Predictive Models**: Models like **Multilayer Perceptron (MLP)**, **Linear Regression**, or **Random Forest** are used to predict outcomes based on input features.
  - **Classification Models**: Models like **Logistic Regression** or **Support Vector Machines (SVM)** classify data into categories.
  - **Clustering Models**: Models like **k-Means** or **DBSCAN** group data into clusters based on similarity.

Key Differences

Aspect	**Feature**	**Method**	**Model**
Definition	A measurable property of the data	A technique or algorithm	A mathematical/computational representation
Role	Input to the model	Processes or analyzes data	Makes predictions or inferences
Examples	RSI, Moving Averages, OBV	LASSO, Pearson's Correlation, PCA	MLP, Linear Regression, Random Forest
Dependency	Independent of methods and models	Operates on features or data	Built using features and methods
Output	None (used as input)	Transformed data or selected features	Predictions or classifications

---

Examples in the Context of the Paper

1. **Feature**:
   - **Example**: The **Relative Strength Index (RSI)** is a feature derived from price data.

   - **Role**: RSI is used as an input to the model to predict stock price movements.
2. **Method**:
   - **Example**: **LASSO** is a method used for feature selection.
   - **Role**: LASSO identifies the most relevant technical indicators (features) by penalizing less important ones.
3. **Model**:
   - **Example**: The **Multilayer Perceptron (MLP)** is a model used for binary classification.
   - **Role**: The MLP takes the selected features (e.g., RSI, Moving Averages) as input and predicts whether the stock price will increase or decrease.

Conclusion

- **Features** are the measurable properties of the data (e.g., technical indicators like RSI).
- **Methods** are the techniques or algorithms used to process or analyze the data (e.g., LASSO for feature selection).
- **Models** are the mathematical or computational structures that use features and methods to make predictions or inferences (e.g., MLP for classification).

By clearly distinguishing between features, methods, and models, researchers can better understand and design effective data analysis workflows, as demonstrated in the paper.

Q4(iii)

In the context of the paper and general data analysis, features can be categorized based on their type, origin, or the information they capture. Here are the main categories of features that I have learned:

---

1. Technical Indicators

These are mathematical calculations based on historical price, volume, or open interest data, used to forecast future price movements. They are commonly used in financial markets and are divided into subcategories:

- **Momentum Indicators**: Measure the speed and strength of price movements (e.g., Relative Strength Index (RSI), Stochastic Oscillator).
- **Trend Indicators**: Identify the direction of the market trend (e.g., Moving Averages, Moving Average Convergence Divergence (MACD)).
- **Volatility Indicators**: Measure price variability (e.g., Bollinger Bands, Average True Range (ATR)).
- **Volume Indicators**: Based on trading volume (e.g., On Balance Volume (OBV), Price Volume Rank (PVR)).
- **Cycle Indicators**: Identify cyclical patterns in price data (e.g., Even Better SineWave (EBSW)).

- **Overlap Indicators**: Overlay on the price chart (e.g., Bollinger Bands, Parabolic SAR).
- **Performance Indicators**: Evaluate the performance of the asset (e.g., Rate of Change (ROC)).

---

2. Fundamental Features

These are derived from the financial health and performance of a company or asset. They include:

- **Financial Ratios**: Such as Price-to-Earnings (P/E) ratio, Debt-to-Equity ratio.
- **Earnings Reports**: Revenue, profit margins, earnings per share (EPS).
- **Economic Indicators**: GDP growth, unemployment rates, inflation rates.

---

3. Statistical Features

These are derived from statistical measures of the data:

- **Central Tendency**: Mean, median, mode.
- **Dispersion**: Variance, standard deviation, range.
- **Correlation**: Pearson's correlation, Spearman's rank correlation.
- **Distribution**: Skewness, kurtosis.

---

4. Temporal Features

These capture time-related information:

- **Time-Based Features**: Day of the week, month, year.
- **Lag Features**: Past values of a variable (e.g., lagged price or volume).
- **Rolling Statistics**: Rolling mean, rolling standard deviation over a time window.

---

5. Textual Features

These are derived from text data:

- **Sentiment Analysis**: Sentiment scores from news articles or social media.
- **Word Counts**: Frequency of specific words or phrases.
- **Topic Modeling**: Identifying topics in text data using techniques like Latent Dirichlet Allocation (LDA).

---

6. Geospatial Features

These capture location-based information:

- **Latitude and Longitude**: Geographic coordinates.

- **Distance Metrics**: Distance between two locations.
- **Regional Indicators**: Country, state, or city-specific data.

---

7. Domain-Specific Features

These are tailored to the specific domain of the data:

- **Healthcare**: Patient age, blood pressure, cholesterol levels.
- **Retail**: Purchase history, customer demographics, product categories.
- **Transportation**: Travel time, route distance, traffic conditions.

---

8. Engineered Features

These are created by transforming or combining existing features:

- **Interaction Terms**: Products or sums of existing features (e.g., price × volume).
- **Polynomial Features**: Squares or cubes of existing features.
- **Aggregated Features**: Summaries of multiple features (e.g., average price over a week).

---

9. Categorical Features

These represent discrete categories or groups:

- **Binary Features**: Yes/No, True/False.
- **Nominal Features**: Categories without order (e.g., colors, countries).
- **Ordinal Features**: Categories with a natural order (e.g., low/medium/high).

---

10. Image Features

These are derived from image data:

- **Pixel Values**: Raw pixel intensities.
- **Edges and Textures**: Detected using techniques like Sobel filters or Gabor filters.
- **Deep Learning Features**: Extracted using convolutional neural networks (CNNs).

Conclusion

The categories of features depend on the type of data and the problem being solved. In the context of the paper, **technical indicators** are the primary category of features, as they are derived from historical stock market data and used to predict future price movements. Understanding these categories helps in selecting, engineering, and optimizing features to build effective predictive models.

Q4(iv)

The authors of the paper employ a systematic approach to optimize the technical indicators used for predicting stock market movements. This process involves selecting the most relevant indicators, reducing dimensionality, and improving the efficiency and accuracy of the predictive model. Here's a detailed breakdown of the optimization process:

---

1. Feature Selection

The authors use a variety of statistical and machine learning techniques to identify the most relevant technical indicators (features) for predicting ETF trends. The goal is to reduce the number of features while retaining the most informative ones. The feature selection methods include:

- **Low Variance**: Removes features with negligible variance, as they provide little information for prediction.
- **Chi-Squared Test**: Evaluates the independence of features from the target variable.
- **LASSO (Least Absolute Shrinkage and Selection Operator)**: A regularization technique that penalizes the absolute size of feature coefficients, effectively performing feature selection.
- **Tree-Based Feature Selection**: Uses ensemble methods like Extra Trees Classifier to rank feature importance.
- **Pearson's Correlation**: Measures the linear relationship between features and the target variable.
- **Principal Feature Analysis**: Retains features that capture maximum variability using principal component analysis (PCA) and k-means clustering.
- **Mean Absolute Difference (MAD)**: Ranks features based on their discriminative power.
- **Dispersion Ratio (DR)**: Measures feature relevance based on the ratio of arithmetic mean to geometric mean.

By applying these methods, the authors identify the top 5% of features (e.g., 10 out of 216 technical indicators) that are most relevant for predicting ETF trends.

---

2. Dimensionality Reduction

- The authors reduce the dimensionality of the dataset by selecting only the most salient features. This not only improves computational efficiency but also helps in avoiding overfitting, where the model performs well on training data but poorly on unseen data.
- The reduced feature set is then used as input to the neural network model, ensuring that the model focuses on the most informative indicators.

---

3. Model Training and Evaluation

- The authors use a **Multilayer Perceptron (MLP)**, a type of neural network, for binary classification (predicting whether the price will increase or decrease).

- The MLP is configured with an adaptive learning rate and early stopping to optimize training and prevent overfitting.
- The model is evaluated using **10-fold cross-validation**, ensuring robust performance estimation by training and testing on different subsets of the data.

Improving Predictive Power

The authors improve the predictive power of the technical indicators through the following steps:

1. **Selecting Relevant Features**:
    - By identifying and retaining only the most relevant features, the model focuses on the indicators that have the strongest relationship with the target variable (price movement direction).
    - This reduces noise and irrelevant information, leading to more accurate predictions.
2. **Reducing Overfitting**:
    - By reducing the number of features, the authors minimize the risk of overfitting, where the model captures noise instead of the underlying pattern.
    - This ensures that the model generalizes well to unseen data.
3. **Enhancing Computational Efficiency**:
    - A smaller set of features reduces the computational complexity of the model, making it faster to train and more scalable for large datasets.
    - This is particularly important for real-time applications like algorithmic trading.
4. **Improving Interpretability**:
    - A reduced feature set makes it easier to interpret the model's predictions and understand the factors driving stock price movements.
    - This is valuable for investors and analysts who need actionable insights.

Importance of Optimizing Technical Indicators for the Neural Network Model

1. **Improved Accuracy**:
    - Optimizing technical indicators ensures that the model uses the most relevant and informative features, leading to more accurate predictions of stock price movements.
2. **Computational Efficiency**:
    - Neural networks, especially deep learning models, can be computationally expensive. By reducing the number of features, the authors significantly decrease the training time and resource requirements.
3. **Generalization**:
    - Optimized features help the model generalize better to unseen data, reducing the risk of overfitting and improving performance on real-world datasets.
4. **Scalability**:
    - A model with fewer features is more scalable and can be applied to larger datasets or real-time trading systems without compromising performance.

5. **Interpretability**:
    - A smaller set of features makes it easier to interpret the model's predictions and understand the factors influencing stock price movements. This is crucial for decision-making in financial markets.

Conclusion

The optimization process of technical indicators involves selecting the most relevant features, reducing dimensionality, and improving the efficiency and accuracy of the predictive model. By systematically identifying and retaining the most informative indicators, the authors enhance the predictive power of the neural network model while ensuring computational efficiency and generalization. This approach is critical for building effective and scalable models for stock market prediction, particularly in the volatile and complex environment of emerging markets.

Q5(i)

Cross-validation is a statistical technique used to evaluate the performance of a predictive model and ensure that it generalizes well to unseen data. It involves partitioning the dataset into multiple subsets, training the model on some subsets, and testing it on the remaining subsets. This process is repeated multiple times, and the results are averaged to provide a robust estimate of the model's performance.

How Cross-Validation Works

1. **Partitioning the Data**:
    - The dataset is divided into **k** equal-sized subsets (or "folds"). For example, in **10-fold cross-validation**, the data is split into 10 subsets.
2. **Training and Testing**:
    - The model is trained **k** times. In each iteration:
        - One subset is held out as the **test set**.
        - The remaining **k-1** subsets are combined to form the **training set**.
        - The model is trained on the training set and evaluated on the test set.
3. **Averaging Results**:
    - The performance metric (e.g., accuracy, precision, recall) is calculated for each iteration.
    - The results from all **k** iterations are averaged to obtain a final performance estimate.

Example of 10-Fold Cross-Validation

1. **Step 1**: Split the dataset into 10 equal parts (folds).
2. **Step 2**: For each fold:
    - Train the model on 9 folds.

    - Test the model on the remaining 1 fold.
3. **Step 3**: Repeat this process 10 times, each time using a different fold as the test set.
4. **Step 4**: Calculate the average performance across all 10 iterations.

Advantages of Cross-Validation

1. **Robust Performance Estimation**:
    - Cross-validation provides a more reliable estimate of model performance compared to a single train-test split, as it uses multiple subsets of the data for testing.
2. **Reduced Overfitting**:
    - By evaluating the model on different subsets of the data, cross-validation helps ensure that the model generalizes well to unseen data and is not overfitted to a specific subset.
3. **Optimal Use of Data**:
    - Cross-validation makes efficient use of the available data, as every data point is used for both training and testing across different iterations.
4. **Model Selection**:
    - Cross-validation can be used to compare different models or hyperparameter settings, helping to identify the best-performing configuration.

Why Cross-Validation is Important in the Paper

In the paper, the authors use **10-fold cross-validation** to evaluate the performance of their neural network model for predicting ETF trends. This approach ensures that:

- The model's performance is robust and not dependent on a specific train-test split.
- The results are generalizable to unseen data, which is critical for real-world applications like stock market prediction.
- The model is optimized for accuracy and computational efficiency, as cross-validation helps identify the best subset of features and model parameters.

Conclusion

Cross-validation is a powerful technique for evaluating and optimizing predictive models. By partitioning the data into multiple subsets and averaging the results, it provides a reliable estimate of model performance and helps ensure that the model generalizes well to new data. In the context of the paper, cross-validation is essential for building an accurate and robust neural network model for predicting stock market trends.

Q5(ii)

k-Fold Cross-Validation is a specific type of cross-validation technique used to evaluate the performance of a predictive model. It involves dividing the dataset into **k** equal-sized subsets (or "folds"), training the model on **k-1** of these subsets, and testing it on the remaining subset. This process is repeated **k** times, with each fold used exactly once as the test set. The results from all **k** iterations are then averaged to provide a robust estimate of the model's performance.

How k-Fold Cross-Validation Works

1. **Partitioning the Data**:
   - The dataset is divided into **k** equal-sized subsets. For example, if **k = 10**, the dataset is split into 10 subsets of approximately the same size.
2. **Training and Testing**:
   - The model is trained **k** times. In each iteration:
     - One of the **k** subsets is held out as the **test set**.
     - The remaining **k-1** subsets are combined to form the **training set**.
     - The model is trained on the training set and evaluated on the test set.
3. **Averaging Results**:
   - The performance metric (e.g., accuracy, precision, recall) is calculated for each iteration.
   - The results from all **k** iterations are averaged to obtain a final performance estimate.

Example of 10-Fold Cross-Validation

1. **Step 1**: Split the dataset into 10 equal parts (folds).
2. **Step 2**: For each fold:
   - Train the model on 9 folds.
   - Test the model on the remaining 1 fold.
3. **Step 3**: Repeat this process 10 times, each time using a different fold as the test set.
4. **Step 4**: Calculate the average performance across all 10 iterations.

Advantages of k-Fold Cross-Validation

1. **Robust Performance Estimation**:
   - k-Fold cross-validation provides a more reliable estimate of model performance compared to a single train-test split, as it uses multiple subsets of the data for testing.
2. **Reduced Overfitting**:
   - By evaluating the model on different subsets of the data, k-Fold cross-validation helps ensure that the model generalizes well to unseen data and is not overfitted to a specific subset.

3. **Optimal Use of Data**:
    - k-Fold cross-validation makes efficient use of the available data, as every data point is used for both training and testing across different iterations.
4. **Model Selection**:
    - k-Fold cross-validation can be used to compare different models or hyperparameter settings, helping to identify the best-performing configuration.

Why k-Fold Cross-Validation is Important in the Paper

In the paper, the authors use **10-fold cross-validation** to evaluate the performance of their neural network model for predicting ETF trends. This approach ensures that:

- The model's performance is robust and not dependent on a specific train-test split.
- The results are generalizable to unseen data, which is critical for real-world applications like stock market prediction.
- The model is optimized for accuracy and computational efficiency, as cross-validation helps identify the best subset of features and model parameters.

Conclusion

k-Fold cross-validation is a powerful technique for evaluating and optimizing predictive models. By partitioning the data into **k** subsets and averaging the results, it provides a reliable estimate of model performance and helps ensure that the model generalizes well to new data. In the context of the paper, k-Fold cross-validation is essential for building an accurate and robust neural network model for predicting stock market trends.

Q5(iii)

The **Jaccard distance** is a metric used to measure the dissimilarity between two sets. It is derived from the **Jaccard similarity coefficient**, which measures the similarity between two sets. The Jaccard distance is defined as **1 minus the Jaccard similarity coefficient**, providing a measure of how different the two sets are.

Jaccard Similarity Coefficient

The **Jaccard similarity coefficient** (or Jaccard index) measures the similarity between two sets by comparing the size of their intersection to the size of their union. The Jaccard similarity coefficient ranges from **0 to 1**, where:

- **1** indicates that the sets are identical.

- **0** indicates that the sets have no elements in common.

Jaccard Distance

The **Jaccard distance** is defined as:

$$\text{Jaccard Distance} = 1 - \text{Jaccard Similarity} = 1 - |A \cap B| \, |A \cup B|$$

- The Jaccard distance ranges from **0 to 1**, where:
  - **0** indicates that the sets are identical.
  - **1** indicates that the sets are completely dissimilar (no overlap).

Interpretation of Jaccard Distance

- **Jaccard Distance = 0**: The sets are identical.
- **Jaccard Distance = 1**: The sets are completely different (no common elements).
- **0 < Jaccard Distance < 1**: The sets share some elements but are not identical. The closer the distance is to 0, the more similar the sets are.

Use of Jaccard Distance in the Paper

In the paper, the authors use the **Jaccard distance** to measure the dissimilarity between the sets of selected features for different ETFs. For example:

- They calculate the Jaccard distance between the feature sets of **ECH** and **EWZ**, **ECH** and **IVV**, and **EWZ** and **IVV**.
- The results show that ETFs from emerging markets (e.g., ECH and EWZ) have a lower Jaccard distance (more similar feature sets) compared to ETFs from developed markets (e.g., ECH and IVV).

This analysis helps the authors understand how the selected features vary across different markets and ETFs.

Advantages of Jaccard Distance

1. **Simplicity**: Easy to compute and interpret.
2. **Normalization**: The distance is normalized between 0 and 1, making it independent of the size of the sets.

3. **Applicability**: Useful for comparing sets of any size, including feature sets, text data, or categorical data.

Conclusion

The **Jaccard distance** is a useful metric for measuring the dissimilarity between two sets. In the context of the paper, it helps the authors compare the feature sets of different ETFs and understand the similarities and differences between emerging and developed markets. By using the Jaccard distance, the authors can ensure that their feature selection process is robust and tailored to the specific characteristics of each ETF.

Q5(iv)

The **Jaccard distance** is one of several distance metrics used to measure dissimilarity between sets or data points. Below, I compare the Jaccard distance to two other commonly used distance metrics: **Euclidean distance** and **Cosine similarity**.

---

1. Jaccard Distance vs. Euclidean Distance

Jaccard Distance

- **Definition**: Measures the dissimilarity between two sets based on the ratio of the size of their intersection to the size of their union.
- **Formula**:
- Jaccard Distance=1−|A∩B||A∪B|
- **Range**: 0 (identical sets) to 1 (completely dissimilar sets).
- **Applicability**: Used for comparing sets, such as feature sets, categorical data, or binary data.

Euclidean Distance

- **Definition**: Measures the straight-line distance between two points in a Euclidean space.
- **Formula**:
- Euclidean Distance=∑i=1n(xi−yi)2
- **Range**: 0 (identical points) to infinity (completely dissimilar points).
- **Applicability**: Used for continuous data, such as numerical feature vectors in a multi-dimensional space.

Key Differences

Aspect	Jaccard Distance	Euclidean Distance
Input	Sets (e.g., feature sets, binary data)	Points in a Euclidean space (continuous data)
Range	0 to 1	0 to infinity
Interpretation	Measures dissimilarity based on set overlap	Measures geometric distance between points
Use Case	Categorical data, feature selection	Numerical data, clustering, regression

---

2. Jaccard Distance vs. Cosine Similarity

Jaccard Distance

- **Definition**: Measures the dissimilarity between two sets based on the ratio of the size of their intersection to the size of their union.
- **Formula**:
- Jaccard Distance=$1-|A\cap B||A\cup B|$
- **Range**: 0 (identical sets) to 1 (completely dissimilar sets).
- **Applicability**: Used for comparing sets, such as feature sets, categorical data, or binary data.

Cosine Similarity

- **Definition**: Measures the cosine of the angle between two vectors in a multi-dimensional space, indicating their orientation similarity.
- **Formula**:
- Cosine Similarity=$A\cdot B\,//\,A\,//\;//\,B\,//$
- **Range**: -1 (completely opposite) to 1 (identical orientation).
- **Applicability**: Used for text data, high-dimensional data, or when the magnitude of vectors is less important than their direction.

Key Differences

Aspect	Jaccard Distance	Cosine Similarity

Input	Sets (e.g., feature sets, binary data)	Vectors (e.g., text embeddings, numerical data)
Range	0 to 1	-1 to 1
Interpretation	Measures dissimilarity based on set overlap	Measures orientation similarity between vectors
Use Case	Categorical data, feature selection	Text analysis, recommendation systems

---

Summary of Comparisons

1. **Jaccard Distance vs. Euclidean Distance**:
   - Jaccard distance is used for **sets** and measures dissimilarity based on **overlap**.
   - Euclidean distance is used for **continuous data** and measures **geometric distance**.
   - Jaccard distance is normalized (0 to 1), while Euclidean distance is unbounded.
2. **Jaccard Distance vs. Cosine Similarity**:
   - Jaccard distance is used for **sets** and measures dissimilarity based on **overlap**.
   - Cosine similarity is used for **vectors** and measures **orientation similarity**.
   - Jaccard distance ranges from 0 to 1, while cosine similarity ranges from -1 to 1.

Conclusion

The **Jaccard distance** is a specialized metric for comparing sets, making it ideal for categorical data or feature selection tasks. In contrast:

- **Euclidean distance** is better suited for continuous data and geometric applications.
- **Cosine similarity** is more appropriate for high-dimensional data or text analysis, where the orientation of vectors matters more than their magnitude.

Each metric has its strengths and is chosen based on the nature of the data and the specific problem being addressed. In the context of the paper, the Jaccard distance is particularly useful for comparing feature sets across different ETFs, as it provides a clear measure of dissimilarity based on shared features.

Q5(v)

The authors define an **optimal solution** as a predictive model that achieves the **best possible balance between accuracy and computational efficiency** while using the **minimum number of relevant features**. Specifically, an optimal solution is characterized by the following criteria:

---

1. High Predictive Accuracy

- The model should accurately predict the direction of stock price movements (up or down) for the selected ETFs.
- The authors measure accuracy as the percentage of correctly predicted price movements during the testing phase.
- The goal is to maximize accuracy while minimizing prediction errors.

---

2. Minimal Feature Set

- The model should use the **smallest possible subset of technical indicators** (features) that still provides high predictive power.
- The authors aim to select only the **top 5% of features** (e.g., 10 out of 216 technical indicators) to reduce dimensionality and computational complexity.
- This ensures that the model is efficient and scalable for real-time applications.

---

3. Computational Efficiency

- The model should be computationally efficient, with reduced training and prediction times.
- By selecting a minimal feature set, the authors achieve an **84.68% reduction in training time**, making the model suitable for real-world applications like algorithmic trading.

---

4. Generalization to Unseen Data

- The model should generalize well to unseen data, avoiding overfitting to the training dataset.
- The authors use **10-fold cross-validation** to ensure that the model's performance is robust and reliable across different subsets of the data.

---

5. Interpretability

- The selected features should be interpretable, providing insights into the factors driving stock price movements.
- This helps investors and analysts understand the model's predictions and make informed decisions.

---

6. Practical Applicability

- The model should provide actionable insights for investors, such as identifying optimal entry and exit points for ETFs.
- It should also assist in **risk management** and **portfolio diversification** by predicting price trends in emerging markets.

How the Authors Achieve an Optimal Solution

1. **Feature Selection**:
   - The authors use a combination of statistical and machine learning techniques (e.g., LASSO, Pearson's correlation, tree-based feature selection) to identify the most relevant technical indicators.
   - This ensures that the model focuses on the features that have the strongest relationship with the target variable (price movement direction).
2. **Model Configuration**:
   - The authors use a **Multilayer Perceptron (MLP)**, a type of neural network, for binary classification.
   - The MLP is configured with an adaptive learning rate and early stopping to optimize training and prevent overfitting.
3. **Cross-Validation**:
   - The authors use **10-fold cross-validation** to evaluate the model's performance and ensure that it generalizes well to unseen data.
4. **Performance Metrics**:
   - The authors measure the model's performance using **accuracy** and **computational efficiency** (training time).
   - They also calculate the **percentage gain** in accuracy achieved by using the optimized feature set compared to the full set of features.

Optimal Solution in Practice

In the study, the authors achieve an optimal solution by:

- Selecting **10 out of 216 technical indicators** (top 5%) as the most relevant features.
- Improving prediction accuracy by approximately **2%** compared to using the full set of features.
- Reducing training time by **84.68%**, making the model computationally efficient.

- Demonstrating robust performance across different ETFs, particularly in emerging markets.

Conclusion

The authors define an **optimal solution** as a predictive model that maximizes accuracy, minimizes computational cost, and generalizes well to unseen data, all while using the smallest possible set of relevant features. By achieving this balance, the model provides a practical and efficient tool for predicting stock price movements in emerging markets, offering valuable insights for investors and analysts.

STEP 1(i)

The financial problem the authors aim to solve with their model is **predicting the direction of stock price movements in emerging markets**. Specifically, the authors focus on forecasting the trends of **Exchange-Traded Funds (ETFs)** in emerging markets, such as the **iShares MSCI Chile ETF (ECH)** and the **iShares MSCI Brazil ETF (EWZ)**, and compare them to a developed market ETF, the **iShares Core S&P 500 ETF (IVV)**.

Key Aspects of the Financial Problem

1. **Emerging Markets Volatility**:
   - Emerging markets are characterized by high volatility, rapid growth, and increased risk compared to developed markets. Predicting price movements in these markets is challenging due to their sensitivity to global economic conditions, political instability, and commodity price fluctuations.
2. **Investor Decision-Making**:
   - Investors in emerging markets need reliable tools to make informed decisions about when to buy or sell assets. Accurate predictions of price trends can help investors maximize returns and minimize losses.
3. **Feature Selection**:
   - The authors address the problem of identifying the most relevant **technical indicators** (features) for predicting ETF trends. By selecting the optimal set of features, the model can improve prediction accuracy and computational efficiency.
4. **Computational Efficiency**:
   - The authors aim to reduce the computational cost of the model by selecting only the most salient features (e.g., 5% of the total technical indicators). This makes the model more practical for real-time applications, such as algorithmic trading.

Practical Implications

The authors' model has several practical implications for investors and financial analysts:

1. **Market Timing**:
   - The model helps investors identify optimal entry and exit points for ETFs in emerging markets, improving their ability to time the market effectively.
2. **Risk Management**:
   - By predicting price trends, the model assists investors in managing risk and reducing exposure to market volatility.
3. **Portfolio Diversification**:
   - The model provides insights into the performance of ETFs in emerging markets, helping investors diversify their portfolios and capitalize on growth opportunities.
4. **Resource Optimization**:
   - The model's focus on feature selection and computational efficiency ensures that it can be applied in real-world scenarios without requiring excessive computational resources.

Conclusion

The financial problem addressed by the authors is **predicting the direction of stock price movements in emerging markets using optimized technical indicators and neural networks**. By solving this problem, the authors provide a valuable tool for investors to make informed decisions, manage risk, and optimize their investment strategies in volatile and rapidly growing markets.

STEP 1(ii)

Predicting stock market movements in **emerging markets** differs significantly from predicting in **developed markets** due to differences in market characteristics, data availability, and external influences. These differences have important implications for the design of predictive models, such as the one proposed in the paper. Below, I outline the key distinctions and explain why they are significant for the model's design.

Key Differences Between Emerging and Developed Markets

1. Market Volatility

- **Emerging Markets**:
  - Characterized by **higher volatility** due to rapid economic growth, political instability, and sensitivity to global commodity prices.
  - Prices can fluctuate dramatically in response to external shocks (e.g., changes in commodity prices, currency fluctuations, or geopolitical events).

- **Developed Markets**:
  - Generally more stable, with lower volatility and more predictable trends.
  - Prices are influenced by well-established economic policies, corporate earnings, and macroeconomic indicators.

Significance for Model Design:

- Models for emerging markets must account for **higher noise and unpredictability** in the data. This requires robust feature selection and regularization techniques to avoid overfitting.
- The model may need to incorporate **external factors** (e.g., commodity prices, currency rates) that significantly impact emerging markets but are less relevant for developed markets.

---

2. Data Availability and Quality

- **Emerging Markets**:
  - Data may be **less reliable or less comprehensive** due to weaker regulatory frameworks and less transparency.
  - Historical data may be shorter or less granular, making it harder to identify long-term trends.
- **Developed Markets**:
  - Data is typically **more reliable, comprehensive, and granular**, with longer historical records and better reporting standards.

Significance for Model Design:

- Models for emerging markets must be designed to handle **noisy or incomplete data**. Techniques like data cleaning, imputation, and robust feature selection are critical.
- The model may need to rely more on **technical indicators** (e.g., price and volume data) rather than fundamental data (e.g., earnings reports), which may be less reliable in emerging markets.

---

3. Market Efficiency

- **Emerging Markets**:
  - Less efficient, with prices often reflecting **lagged or incomplete information**.
  - Opportunities for **arbitrage** or outperformance may exist due to market inefficiencies.
- **Developed Markets**:
  - More efficient, with prices reflecting all available information more quickly.
  - Predictions are harder to make because prices adjust rapidly to new information.

Significance for Model Design:

- Models for emerging markets can exploit **inefficiencies** in the market, making technical analysis and short-term predictions more viable.
- In developed markets, the model may need to focus on **long-term trends** or incorporate more sophisticated techniques (e.g., sentiment analysis, news data) to capture subtle price movements.

---

4. External Influences

- **Emerging Markets**:
  - Highly sensitive to **external factors** such as global commodity prices, foreign investment flows, and currency exchange rates.
  - Political instability and regulatory changes can have a significant impact on market performance.
- **Developed Markets**:
  - Less sensitive to external shocks, with prices driven more by **domestic economic conditions** and corporate performance.

Significance for Model Design:

- Models for emerging markets must incorporate **external variables** (e.g., commodity prices, currency rates) that are less relevant for developed markets.
- The model may need to be **adaptive** to sudden changes in external conditions, requiring techniques like online learning or dynamic feature selection.

---

5. Investor Behavior

- **Emerging Markets**:
  - Investor behavior may be more **speculative** and driven by short-term trends.
  - Retail investors may play a larger role, leading to higher market sentiment-driven volatility.
- **Developed Markets**:
  - Investor behavior is typically more **institutional** and driven by long-term fundamentals.
  - Markets are less influenced by sentiment and more by rational analysis.

Significance for Model Design:

- Models for emerging markets may need to incorporate **sentiment analysis** or behavioral data to capture the impact of retail investor behavior.
- In developed markets, the model can focus more on **fundamental analysis** and long-term trends.

Why the Distinction is Significant for the Model's Design

1. **Feature Selection**:
   - The model must prioritize different types of features for emerging and developed markets. For example:
     - Emerging markets: Focus on **technical indicators** and **external factors** (e.g., commodity prices, currency rates).
     - Developed markets: Incorporate **fundamental data** (e.g., earnings reports, macroeconomic indicators).
2. **Robustness to Noise**:
   - The model must be designed to handle **noisy and incomplete data** in emerging markets, requiring robust preprocessing and feature selection techniques.
3. **Adaptability**:

    - The model must be **adaptive** to sudden changes in external conditions, which are more common in emerging markets.
4. **Computational Efficiency**:
    - The model must be computationally efficient, especially for emerging markets, where real-time predictions are critical due to higher volatility.
5. **Interpretability**:
    - The model should provide interpretable results, as investors in emerging markets may need more transparency to trust the predictions.

Conclusion

Predicting stock market movements in **emerging markets** differs significantly from predicting in **developed markets** due to differences in volatility, data quality, market efficiency, external influences, and investor behavior. These distinctions are critical for the design of predictive models, as they influence the choice of features, robustness to noise, adaptability, computational efficiency, and interpretability. By addressing these differences, the authors' model is better equipped to provide accurate and actionable predictions for investors in both emerging and developed markets.

STEP 2 (i)

The results of the paper provide several key insights into the effectiveness of the authors' approach for predicting stock market movements in emerging markets. Below are the main takeaways:

---

1. Feature Selection Improves Model Performance

- The authors demonstrate that **selecting the most relevant technical indicators** significantly improves the predictive power of the model.
- By reducing the number of features to **5% of the total technical indicators** (e.g., 10 out of 216), the model achieves:
    - **Higher Accuracy**: The accuracy of the model improves by approximately **2%** compared to using the full set of features.
    - **Reduced Computational Cost**: Training time is reduced by **84.68%**, making the model more efficient and scalable for real-time applications.

---

2. Emerging Markets vs. Developed Markets

- The model performs differently for **emerging markets** (e.g., ECH, EWZ) and **developed markets** (e.g., IVV):

- **Emerging Markets**: The model achieves higher accuracy for ETFs in emerging markets, likely due to the **higher volatility** and **inefficiencies** in these markets, which create more predictable patterns.
- **Developed Markets**: The model's performance is slightly lower for developed markets, as these markets are more efficient and less influenced by short-term trends.

---

3. Key Technical Indicators

- The authors identify the **most salient technical indicators** for predicting ETF trends. These include:
  - **Momentum Indicators**: Such as Relative Strength Index (RSI) and Stochastic Oscillator.
  - **Trend Indicators**: Such as Moving Averages and Bollinger Bands.
  - **Volume Indicators**: Such as On Balance Volume (OBV) and Price Volume Rank (PVR).
- The selected features vary across ETFs, reflecting the unique characteristics of each market.

---

4. Jaccard Distance Analysis

- The authors use the **Jaccard distance** to compare the feature sets of different ETFs:
  - **Emerging Markets**: ETFs from emerging markets (e.g., ECH and EWZ) have a **lower Jaccard distance**, indicating more similar feature sets.
  - **Developed Markets**: ETFs from developed markets (e.g., IVV) have a **higher Jaccard distance** compared to emerging markets, reflecting differences in market behavior and feature relevance.

---

5. Practical Implications for Investors

- The model provides actionable insights for investors in emerging markets:
  - **Market Timing**: Helps investors identify optimal entry and exit points for ETFs.
  - **Risk Management**: Assists in managing risk by predicting price trends and reducing exposure to volatility.
  - **Portfolio Diversification**: Provides insights into the performance of ETFs in emerging markets, helping investors diversify their portfolios.

---

6. Computational Efficiency

- The model's focus on **feature selection** and **computational efficiency** ensures that it can be applied in real-world scenarios without requiring excessive computational resources.
- This makes the model suitable for **real-time applications**, such as algorithmic trading, where speed and accuracy are critical.

Conclusion

The main takeaways of the results are:

1. **Feature selection** improves model accuracy and computational efficiency.
2. The model performs better for **emerging markets** due to their higher volatility and inefficiencies.
3. **Key technical indicators** vary across ETFs, reflecting differences in market behavior.
4. The **Jaccard distance** analysis highlights similarities and differences between emerging and developed markets.
5. The model provides **practical benefits** for investors, including market timing, risk management, and portfolio diversification.
6. The model is **computationally efficient**, making it suitable for real-time applications.

These takeaways demonstrate the effectiveness of the authors' approach and its potential to provide valuable insights for investors in emerging markets.

STEP 2 (ii)

The authors of the paper identified a subset of **technical indicators** (features) that were particularly useful for predicting the direction of stock price movements in emerging markets. These features were selected using various statistical and machine learning techniques, such as LASSO, Pearson's correlation, and tree-based feature selection. Below are the **specific features** that stood out as useful in the study:

---

1. Momentum Indicators

Momentum indicators measure the speed and strength of price movements and are critical for identifying trends and potential reversals. The following momentum indicators were found to be useful:

- **Relative Strength Index (RSI)**:
  - Measures whether a stock is overbought or oversold.
  - Useful for identifying potential reversal points.
- **Stochastic Oscillator (STOCH)**:
  - Compares a stock's closing price to its price range over a specific period.
  - Helps identify momentum and potential turning points.
- **Williams %R (WILLR)**:
  - Similar to the Stochastic Oscillator but focuses on overbought and oversold conditions.
  - Useful for short-term trading strategies.

---

2. Trend Indicators

Trend indicators help identify the direction of the market trend and are essential for understanding whether prices are moving upward, downward, or sideways. The following trend indicators were useful:

- **Moving Averages (MA)**:
  - Smooths out price data to identify trends.
  - Examples include Simple Moving Average (SMA) and Exponential Moving Average (EMA).
- **Bollinger Bands Percent (BBP)**:
  - Measures the price relative to the upper and lower Bollinger Bands.
  - Useful for identifying volatility and potential breakout points.
- **Correlation Trend Indicator (CTI)**:
  - Measures the correlation between price and a trend line.
  - Helps confirm the strength of a trend.

---

3. Volume Indicators

Volume indicators provide insights into the strength of price movements based on trading volume. The following volume indicators were useful:

- **On Balance Volume (OBV)**:
  - Tracks cumulative buying and selling pressure based on volume.
  - Useful for confirming price trends.
- **Price Volume Rank (PVR)**:
  - Ranks the relationship between price changes and trading volume.
  - Helps identify strong or weak price movements.

---

4. Cycle Indicators

Cycle indicators identify cyclical patterns in price data, which are particularly relevant in emerging markets. The following cycle indicator was useful:

- **Even Better SineWave (EBSW)**:
  - Identifies market cycles using a low-pass filter to remove noise.
  - Useful for predicting turning points in price trends.

---

5. Statistical Indicators

Statistical indicators provide insights into the distribution and variability of price data. The following statistical indicator was useful:

- **Z-score (ZS)**:
  - Normalizes price data using the mean and standard deviation.
  - Useful for identifying outliers and extreme price movements.

---

6. Utility Indicators

Utility indicators are helper functions or transformations that enhance the analysis. The following utility indicator was useful:

- **Decreasing (DEC)** and **Increasing (INC)**:
  - Boolean indicators that identify whether the price is decreasing or increasing compared to the previous day.
  - Useful for capturing short-term price movements.

---

7. Volatility Indicators

Volatility indicators measure the variability of price movements, which is critical in volatile emerging markets. The following volatility indicator was useful:

- **Average True Range (ATR)**:
  - Measures market volatility by calculating the average range between high and low prices.
  - Useful for setting stop-loss levels and assessing risk.

Summary of Useful Features

Category	Useful Features
Momentum	RSI, Stochastic Oscillator (STOCH), Williams %R (WILLR)
Trend	Moving Averages (MA), Bollinger Bands Percent (BBP), Correlation Trend Indicator (CTI)
Volume	On Balance Volume (OBV), Price Volume Rank (PVR)
Cycle	Even Better SineWave (EBSW)
Statistical	Z-score (ZS)
Utility	Decreasing (DEC), Increasing (INC)
Volatility	Average True Range (ATR)

Why These Features Are Useful

1. **Relevance to Emerging Markets**:
   - Emerging markets are characterized by **high volatility** and **cyclical trends**, making momentum, trend, and cycle indicators particularly useful.
   - Volume indicators help confirm the strength of price movements, which is critical in markets driven by speculative trading.
2. **Improved Predictive Power**:
   - These features capture the **key dynamics** of price movements, such as trends, reversals, and volatility, enabling the model to make accurate predictions.
3. **Computational Efficiency**:

- By selecting only the most relevant features, the model reduces computational complexity while maintaining high accuracy.

Conclusion

The study identified a set of **specific technical indicators** that are particularly useful for predicting stock price movements in emerging markets. These features include momentum indicators (e.g., RSI, Stochastic Oscillator), trend indicators (e.g., Moving Averages, Bollinger Bands), volume indicators (e.g., OBV, PVR), and cycle indicators (e.g., EBSW). By focusing on these features, the authors' model achieves higher accuracy and computational efficiency, making it a valuable tool for investors in emerging markets.

STEP 3(a)

For the purpose of replication, I will focus on the **iShares MSCI Chile ETF (ECH)**. This fund represents an emerging market and provides an interesting case study due to its sensitivity to commodity prices, political factors, and economic conditions in Chile. Below, I outline the steps and considerations for replicating the study for ECH.

Why Choose ECH?

1. **Emerging Market Characteristics**:
   - ECH is heavily influenced by Chile's economy, which is driven by copper exports and other natural resources.
   - The fund exhibits higher volatility compared to developed market ETFs, making it a challenging but rewarding target for predictive modeling.
2. **Data Availability**:
   - Historical price and volume data for ECH are readily available from sources like Yahoo Finance, making it feasible to replicate the study.
3. **Practical Implications**:
   - Predicting trends for ECH can provide valuable insights for investors looking to capitalize on opportunities in emerging markets.

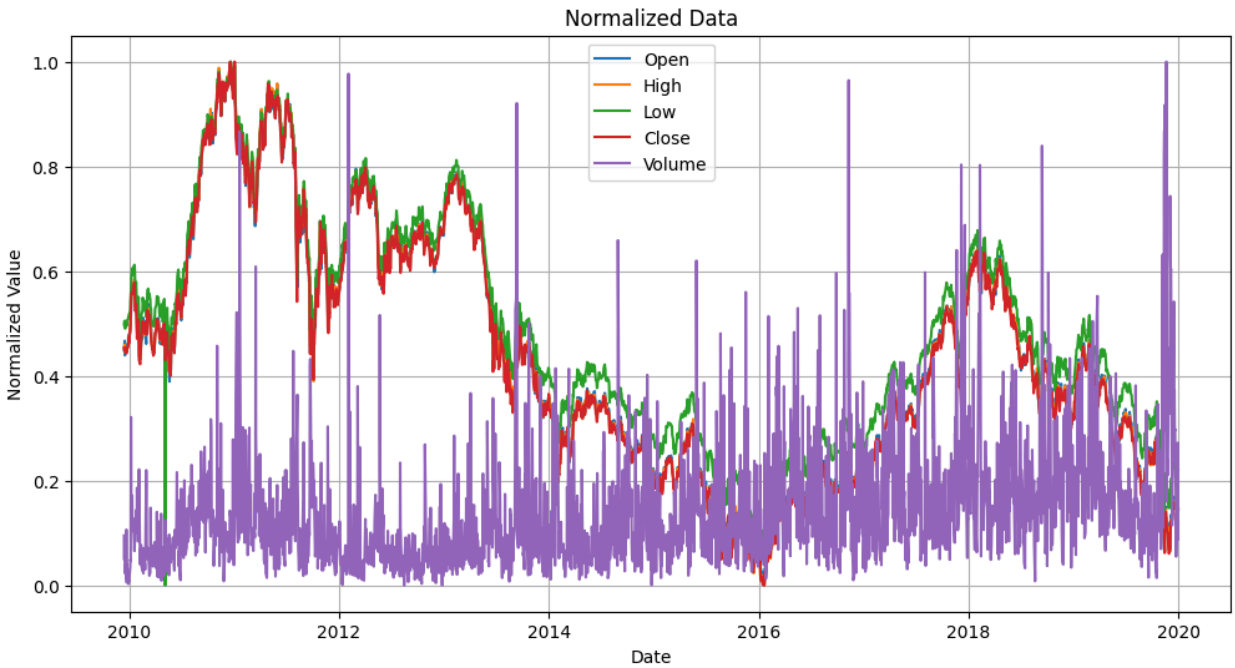
STEP 3(b)

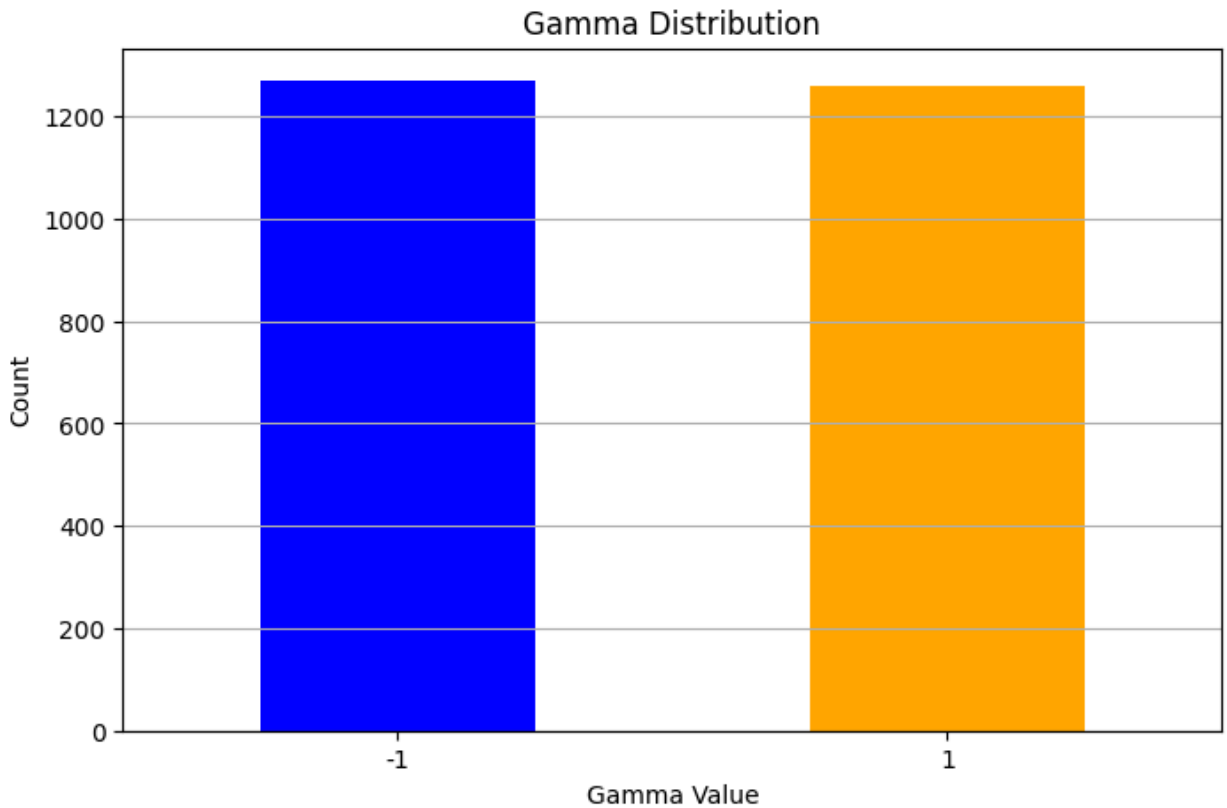
[*********100%***********] 1 of 1 completedRaw data downloaded and saved to ECH_historical_data.csv
Column names in the dataset: ['Close', 'High', 'Low', 'Open', 'Volume']

Preprocessed data saved to ECH_preprocessed_data.csv

	Close	High	Low	Open	Volume	Gamma
Date						
2009-12-14	0.453346	0.454779	0.497858	0.448796	0.095145	1
2009-12-15	0.449461	0.450883	0.504424	0.451845	0.048906	1
2009-12-16	0.450687	0.459496	0.505362	0.467086	0.089609	-1
2009-12-17	0.446599	0.445550	0.492230	0.439448	0.023376	1
2009-12-18	0.450279	0.449447	0.490353	0.457129	0.040088	-1
2009-12-21	0.454572	0.451908	0.499921	0.450828	0.014354	1
2009-12-22	0.443187	0.449749	0.500354	0.457428	0.106423	-1
2009-12-23	0.447495	0.446662	0.495836	0.454369	0.005639	1
2009-12-24	0.449341	0.452424	0.506190	0.460079	0.040139	-1
2009-12-28	0.456316	0.456746	0.508072	0.456816	0.010612	1

[*********************100%***********************] 1 of 1 completed
Raw data downloaded and saved to ECH_historical_data.csv
Preprocessed data saved to ECH_preprocessed_data.csv





Technical indicators computed and saved to ECH_data_with_indicators.csv

Close High Low Open Volume Gamma RSI_14 \
Date

2009-12-14	0.453346	0.454779	0.497858	0.448796	0.095145	1	NaN
2009-12-15	0.449461	0.450883	0.504424	0.451845	0.048906	1	NaN
2009-12-16	0.450687	0.459496	0.505362	0.467086	0.089609	-1	NaN
2009-12-17	0.446599	0.445550	0.492230	0.439448	0.023376	1	NaN
2009-12-18	0.450279	0.449447	0.490353	0.457129	0.040088	-1	NaN
2009-12-21	0.454572	0.451908	0.499921	0.450828	0.014354	1	NaN
2009-12-22	0.443187	0.449749	0.500354	0.457428	0.106423	-1	NaN
2009-12-23	0.447495	0.446662	0.495836	0.454369	0.005639	1	NaN
2009-12-24	0.449341	0.452424	0.506190	0.460079	0.040139	-1	NaN
2009-12-28	0.456316	0.456746	0.508072	0.456816	0.010612	1	NaN

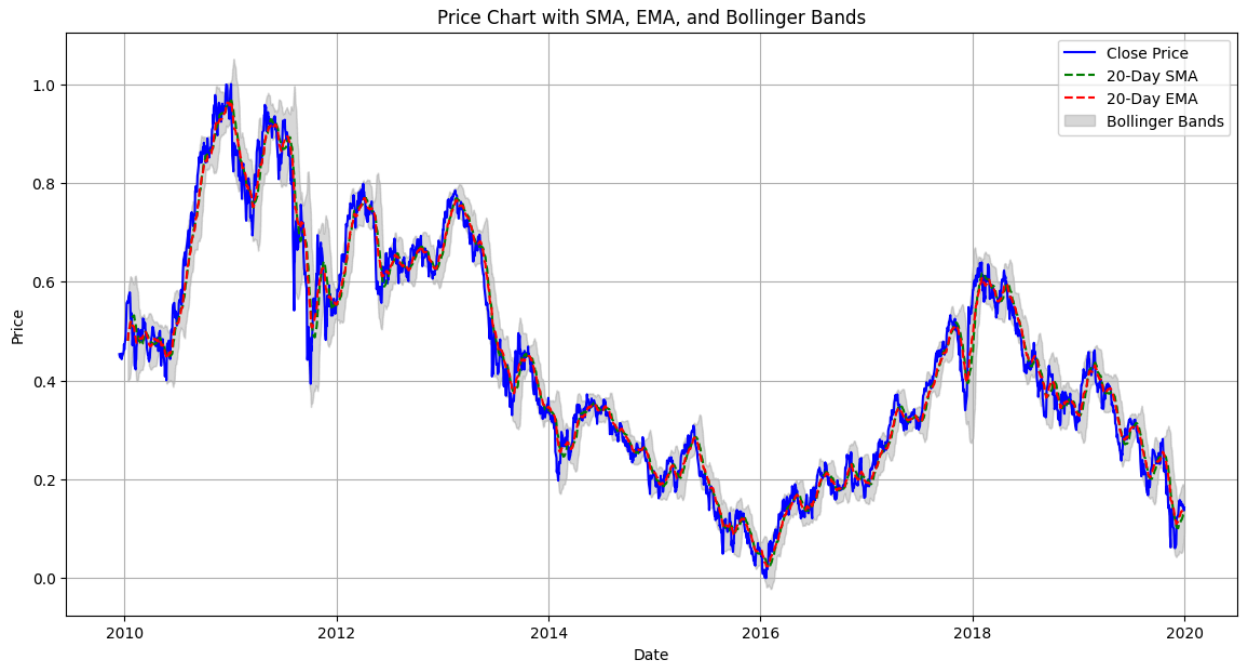
	SMA_20	EMA_20	BBL_20_2.0	...	BBU_20_2.0	BBB_20_2.0 \
Date				...		
2009-12-14	NaN	NaN	NaN	...	NaN	NaN
2009-12-15	NaN	NaN	NaN	...	NaN	NaN
2009-12-16	NaN	NaN	NaN	...	NaN	NaN
2009-12-17	NaN	NaN	NaN	...	NaN	NaN
2009-12-18	NaN	NaN	NaN	...	NaN	NaN
2009-12-21	NaN	NaN	NaN	...	NaN	NaN
2009-12-22	NaN	NaN	NaN	...	NaN	NaN
2009-12-23	NaN	NaN	NaN	...	NaN	NaN
2009-12-24	NaN	NaN	NaN	...	NaN	NaN
2009-12-28	NaN	NaN	NaN	...	NaN	NaN

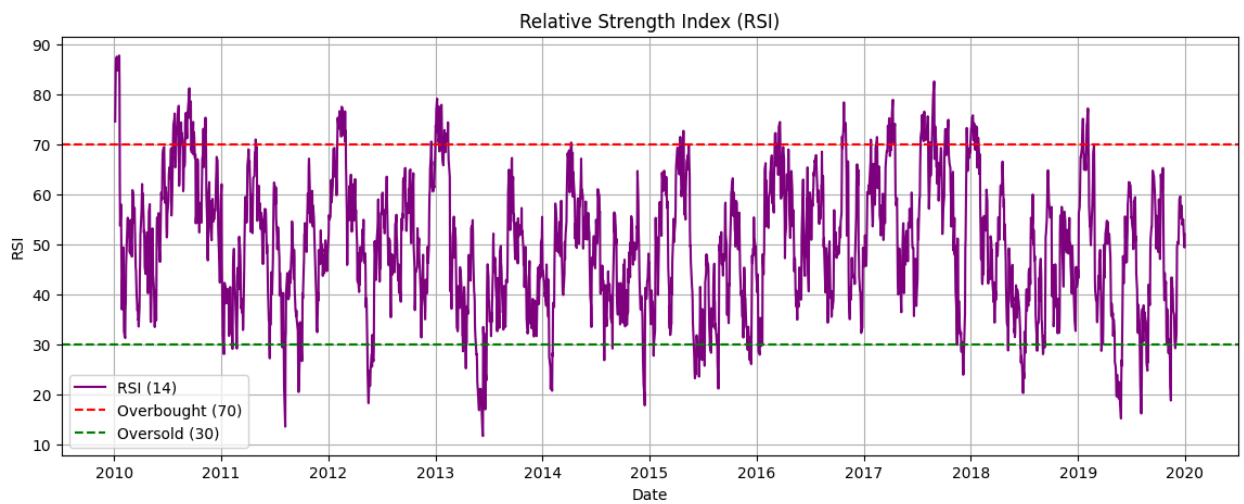
	BBP_20_2.0	MACD_12_26_9	MACDh_12_26_9	MACDs_12_26_9	OBV \
Date					
2009-12-14	NaN	NaN	NaN	NaN	0.095145
2009-12-15	NaN	NaN	NaN	NaN	0.046240
2009-12-16	NaN	NaN	NaN	NaN	0.135849
2009-12-17	NaN	NaN	NaN	NaN	0.112472
2009-12-18	NaN	NaN	NaN	NaN	0.152561
2009-12-21	NaN	NaN	NaN	NaN	0.166914
2009-12-22	NaN	NaN	NaN	NaN	0.060491
2009-12-23	NaN	NaN	NaN	NaN	0.066130
2009-12-24	NaN	NaN	NaN	NaN	0.106270
2009-12-28	NaN	NaN	NaN	NaN	0.116881

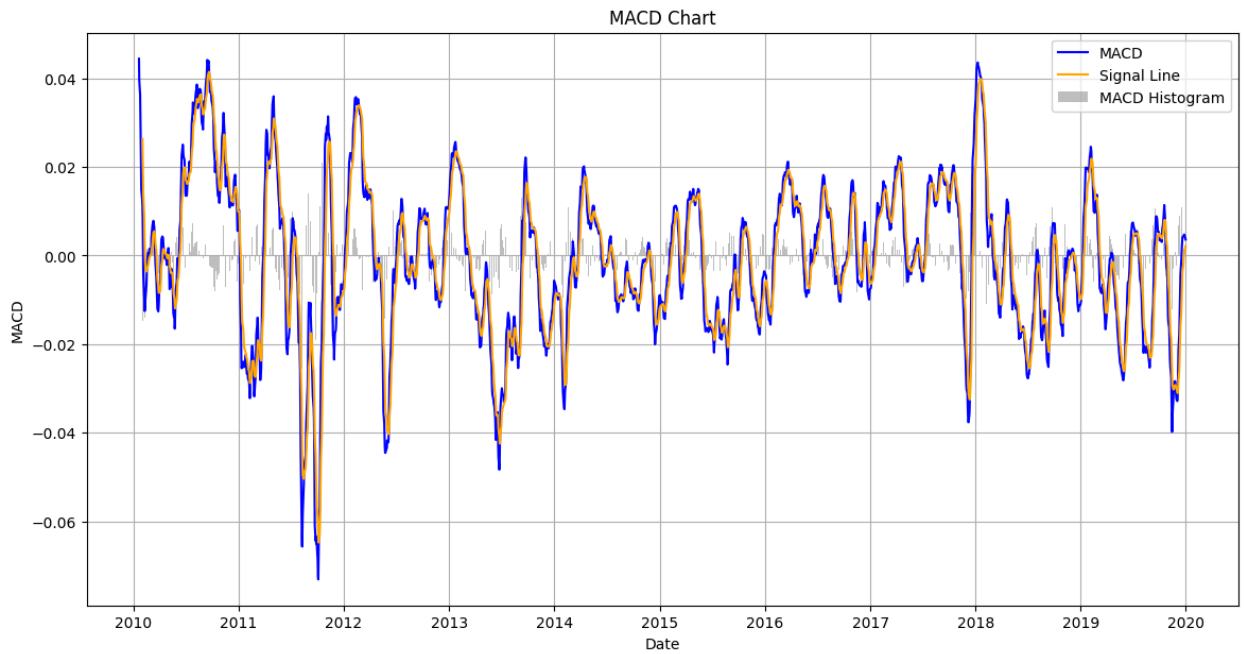
	STOCHk_14_3_3	STOCHd_14_3_3	ATRr_14
Date			
2009-12-14	NaN	NaN	NaN
2009-12-15	NaN	NaN	NaN
2009-12-16	NaN	NaN	NaN
2009-12-17	NaN	NaN	NaN
2009-12-18	NaN	NaN	NaN

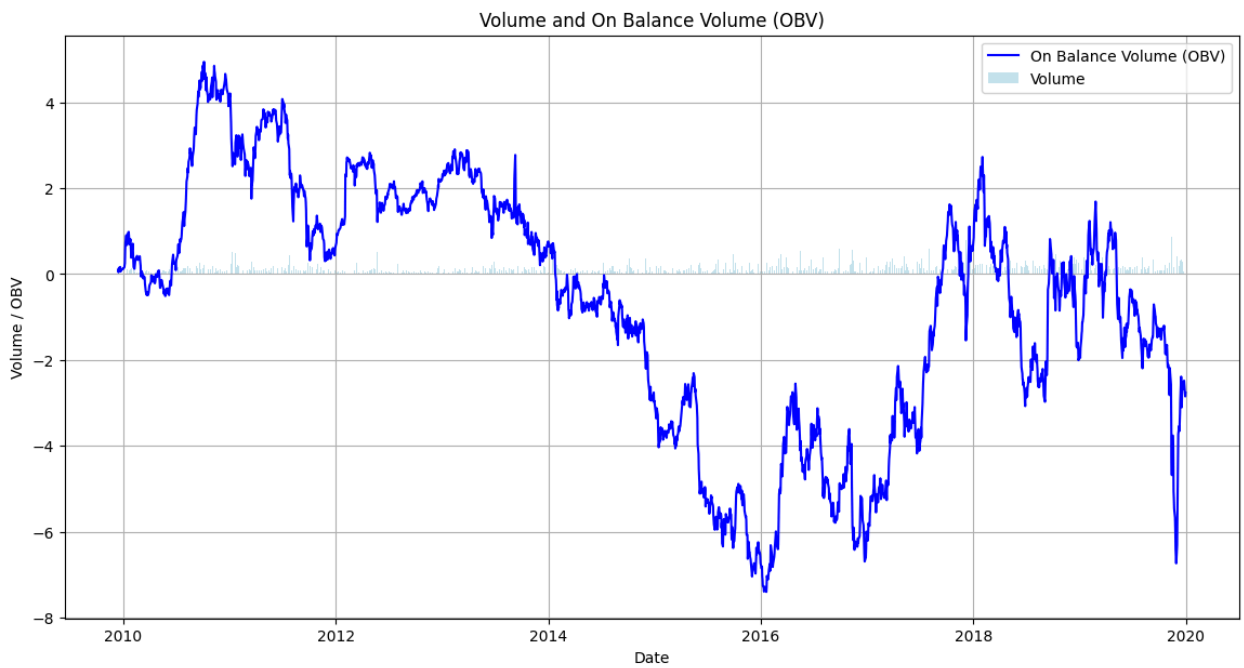
2009-12-21	NaN	NaN	NaN
2009-12-22	NaN	NaN	NaN
2009-12-23	NaN	NaN	NaN
2009-12-24	NaN	NaN	NaN
2009-12-28	NaN	NaN	NaN

[10 rows x 21 columns]









Missing values in the dataset:

Close	0
High	0
Low	0
Open	0
Volume	0
RSI_14	14
SMA_20	19

```
EMA_20          19
BBL_20_2.0      19
BBM_20_2.0      19
BBU_20_2.0      19
BBB_20_2.0      19
BBP_20_2.0      19
MACD_12_26_9    25
MACDh_12_26_9   33
MACDs_12_26_9   33
OBV             0
STOCHk_14_3_3   15
STOCHd_14_3_3   17
ATRr_14         14
dtype: int64
Selected features saved to ECH_selected_features.csv
```

	SMA_20	MACD_12_26_9	BBB_20_2.0	Close	MACDs_12_26_9 \
Date					
2009-12-14	NaN	NaN	NaN	0.453346	NaN
2009-12-15	NaN	NaN	NaN	0.449461	NaN
2009-12-16	NaN	NaN	NaN	0.450687	NaN
2009-12-17	NaN	NaN	NaN	0.446599	NaN
2009-12-18	NaN	NaN	NaN	0.450279	NaN
2009-12-21	NaN	NaN	NaN	0.454572	NaN
2009-12-22	NaN	NaN	NaN	0.443187	NaN
2009-12-23	NaN	NaN	NaN	0.447495	NaN
2009-12-24	NaN	NaN	NaN	0.449341	NaN
2009-12-28	NaN	NaN	NaN	0.456316	NaN

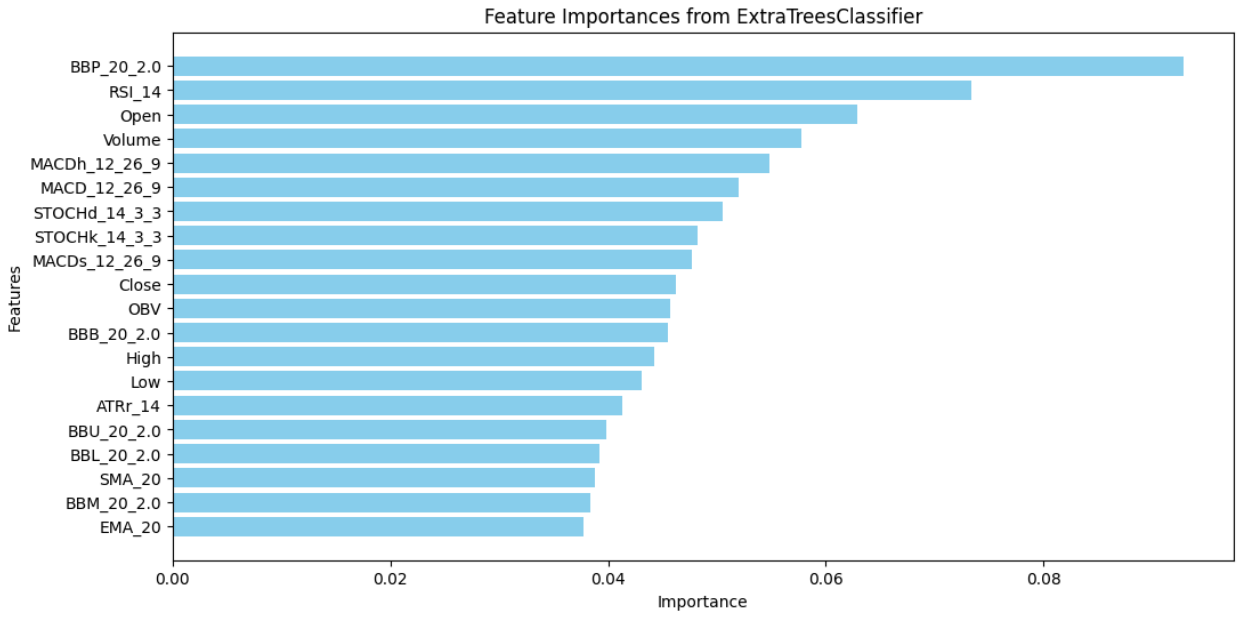
	MACDh_12_26_9	Open	OBV	STOCHd_14_3_3	STOCHk_14_3_3 \
Date					
2009-12-14	NaN	0.448796	0.095145	NaN	NaN
2009-12-15	NaN	0.451845	0.046240	NaN	NaN
2009-12-16	NaN	0.467086	0.135849	NaN	NaN
2009-12-17	NaN	0.439448	0.112472	NaN	NaN
2009-12-18	NaN	0.457129	0.152561	NaN	NaN
2009-12-21	NaN	0.450828	0.166914	NaN	NaN
2009-12-22	NaN	0.457428	0.060491	NaN	NaN
2009-12-23	NaN	0.454369	0.066130	NaN	NaN

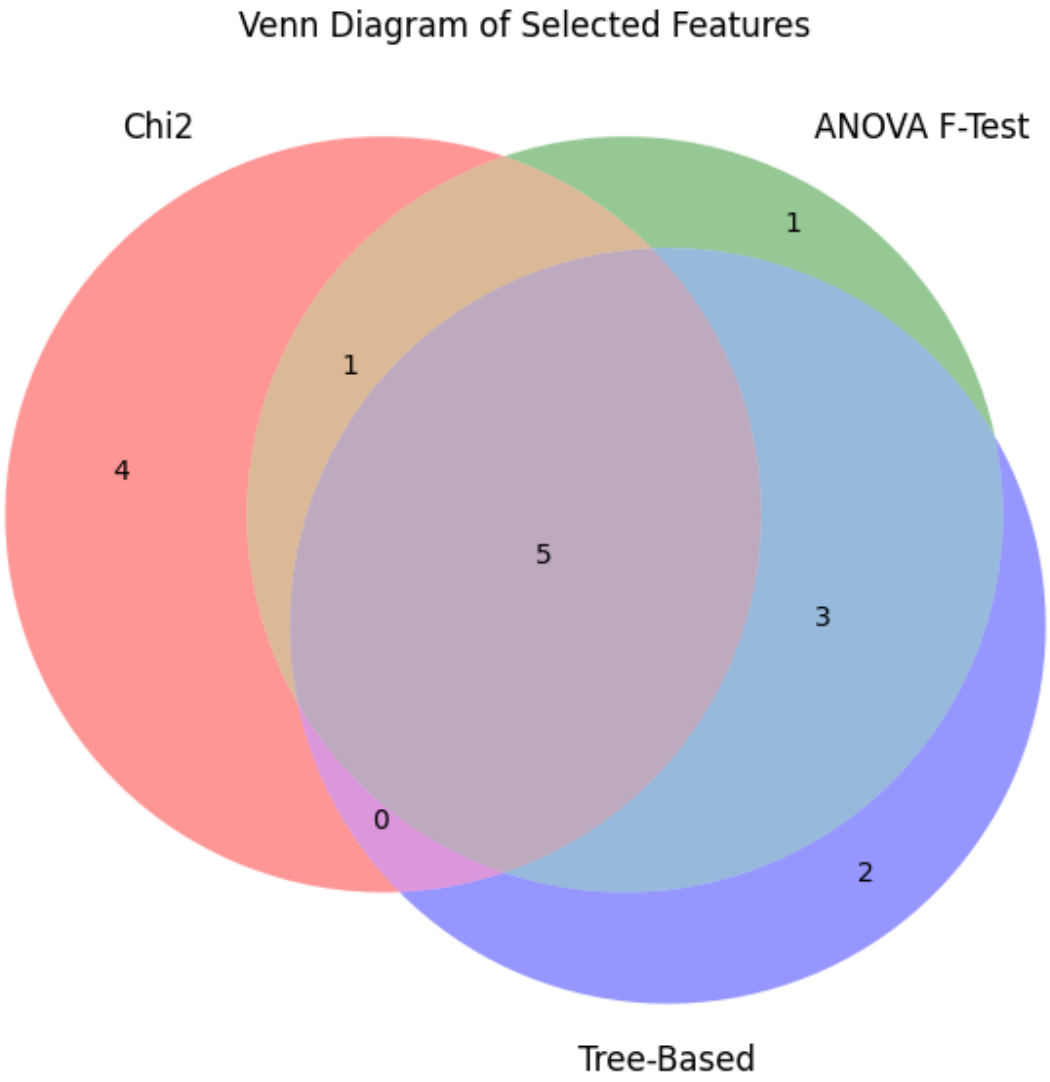
2009-12-24	**NaN**	**0.460079**	**0.106270**	**NaN**	**NaN**
2009-12-28	**NaN**	**0.456816**	**0.116881**	**NaN**	**NaN**

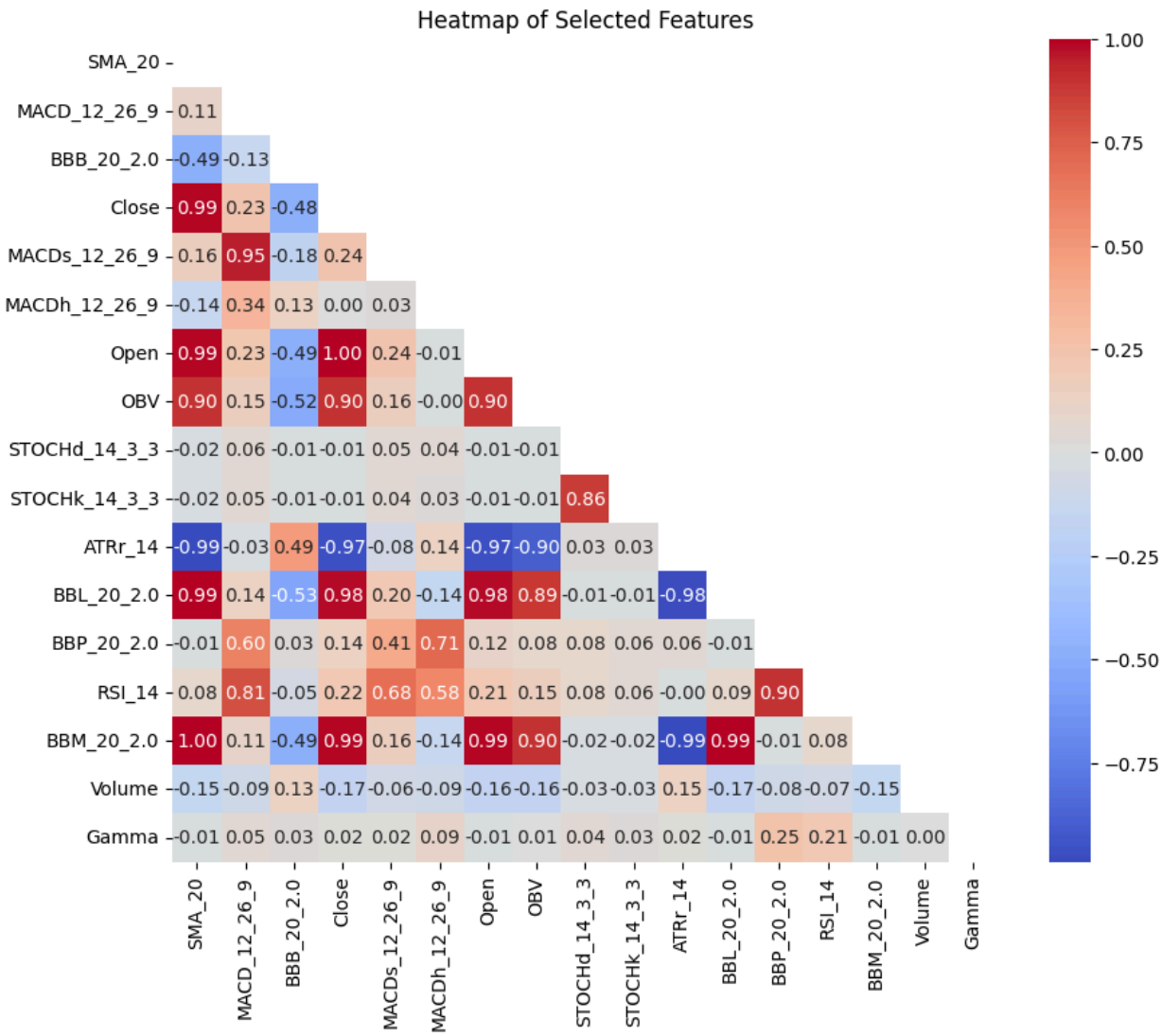
	ATRr_14	**BBL_20_2.0**	**BBP_20_2.0**	**RSI_14**	**BBM_20_2.0**	**Volume **
Date						
2009-12-14	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.095145**
2009-12-15	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.048906**
2009-12-16	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.089609**
2009-12-17	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.023376**
2009-12-18	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.040088**
2009-12-21	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.014354**
2009-12-22	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.106423**
2009-12-23	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.005639**
2009-12-24	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.040139**
2009-12-28	**NaN**	**NaN**	**NaN**	**NaN**	**NaN**	**0.010612**

	Gamma
Date	
2009-12-14	**1**
2009-12-15	**1**
2009-12-16	**-1**
2009-12-17	**1**
2009-12-18	**-1**

2009-12-21	**1**
2009-12-22	**-1**
2009-12-23	**1**
2009-12-24	**-1**
2009-12-28	**1**



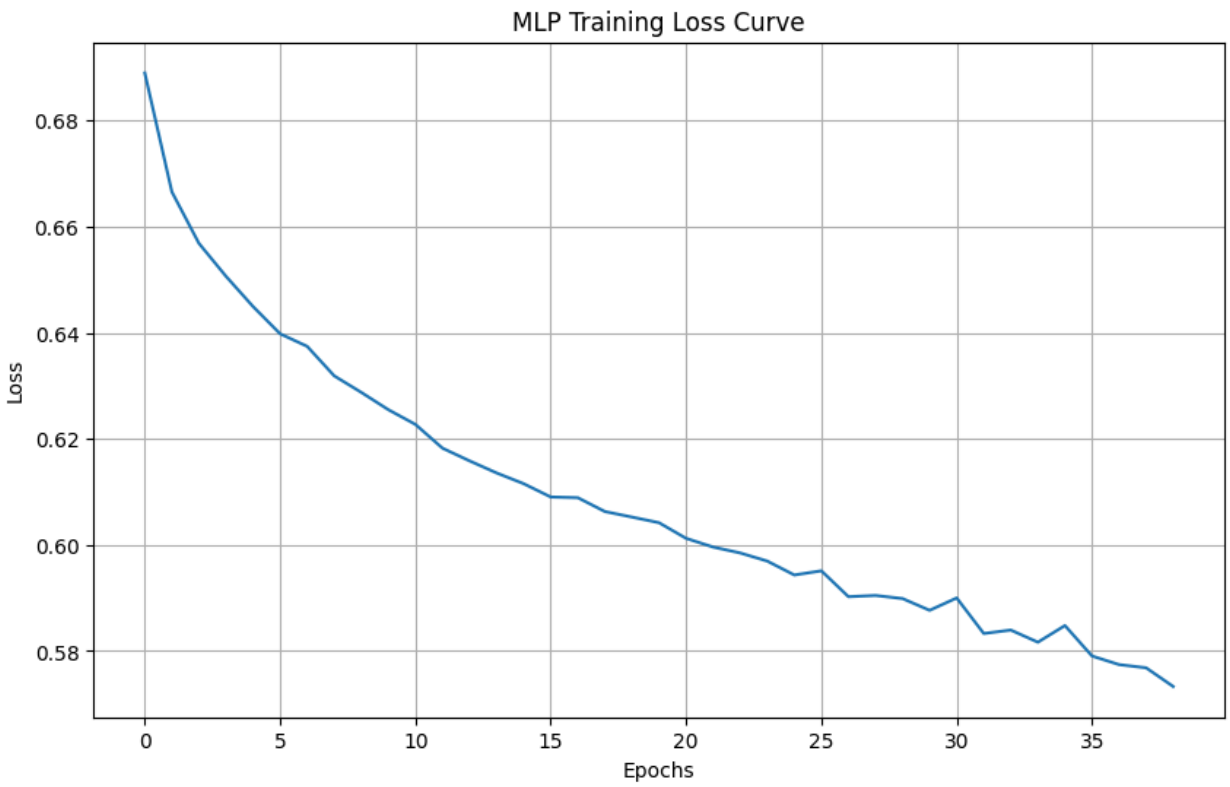

Venn Diagram of Selected Features
Chi2
ANOVA F-Test
Tree-Based
4
1
1
5
3
0
2

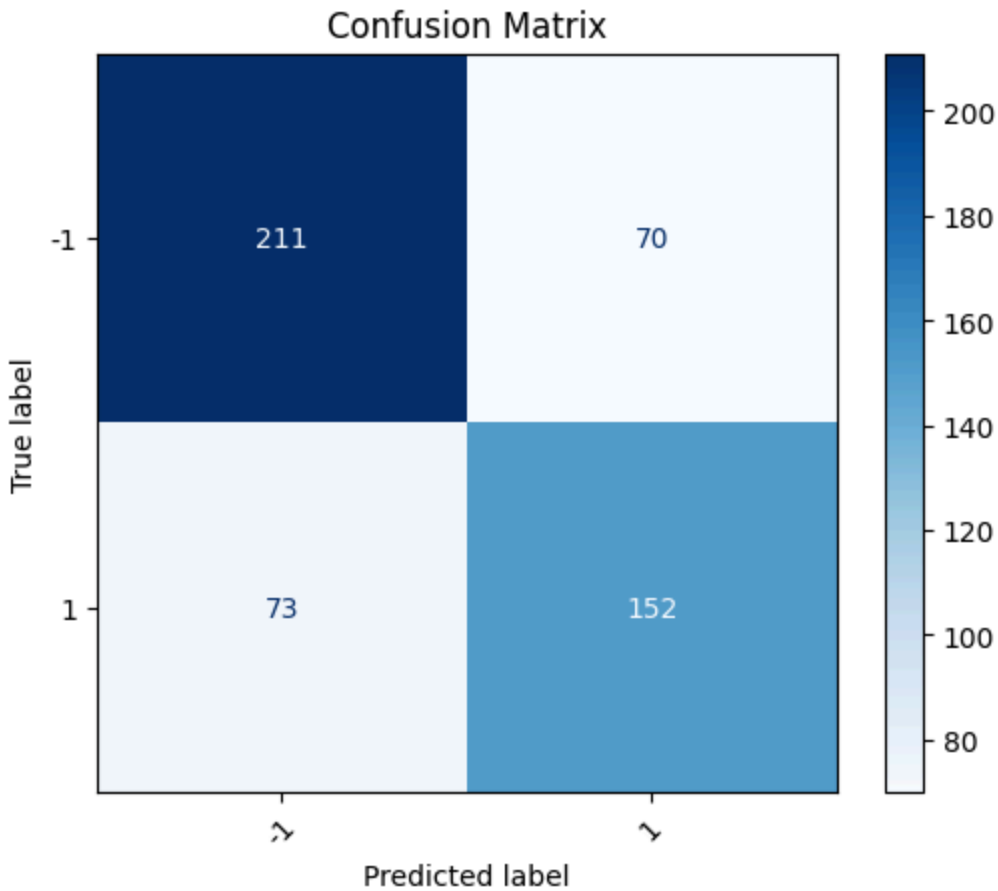


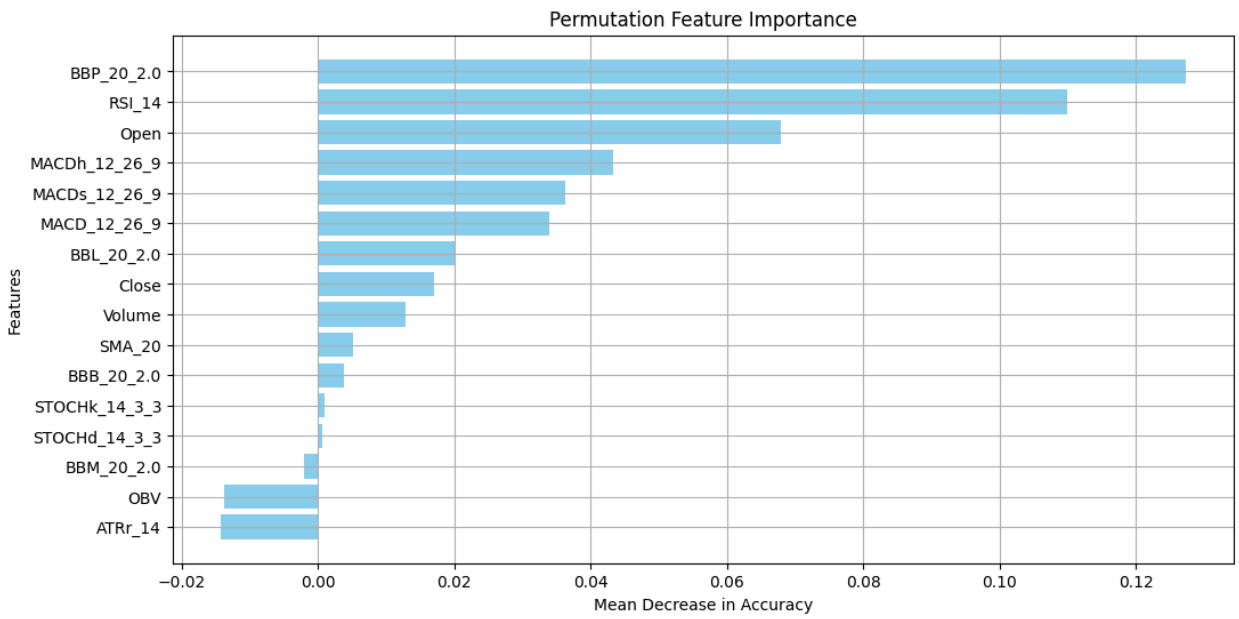
Model Accuracy: 71.74%
Classification Report:

	precision	recall	f1-score	support
-1	0.74	0.75	0.75	281
1	0.68	0.68	0.68	225
accuracy			0.72	506
macro avg	0.71	0.71	0.71	506
weighted avg	0.72	0.72	0.72	506

	precision	recall	f1-score	support
-1	0.742958	0.750890	0.746903	281.000000
1	0.684685	0.675556	0.680089	225.000000
accuracy	0.717391	0.717391	0.717391	0.717391
macro avg	0.713821	0.713223	0.713496	506.000000
weighted avg	0.717046	0.717391	0.717193	506.000000




Confusion Matrix
211
70
73
152
-1
1
True label
Predicted label
200
180
160
140
120
100
80


Permutation Feature Importance
BBP_20_2.0
RSI_14
Open
MACDh_12_26_9
MACDs_12_26_9
MACD_12_26_9
BBL_20_2.0
Close
Volume
SMA_20
BBB_20_2.0
STOCHk_14_3_3
STOCHd_14_3_3
BBM_20_2.0
OBV
ATRr_14
Features
−0.02
0.00
0.02
0.04
0.06
0.08
0.10
0.12
Mean Decrease in Accuracy

Step 3(c)

Top 10 Features by Correlation with Gamma:

Gamma 1.000000

BBP_20_2.0 0.251631

RSI_14 0.206282

MACDh_12_26_9 0.093332

MACD_12_26_9 0.045117

STOCHd_14_3_3 0.039653

STOCHk_14_3_3 0.031081

BBB_20_2.0 0.025680

ATRr_14 0.020945

MACDs_12_26_9 0.020146

Name: Gamma, dtype: float64

Selected Features:

Index(['BBP_20_2.0', 'RSI_14', 'MACDh_12_26_9', 'MACD_12_26_9',

'STOCHd_14_3_3', 'STOCHk_14_3_3', 'BBB_20_2.0', 'ATRr_14',

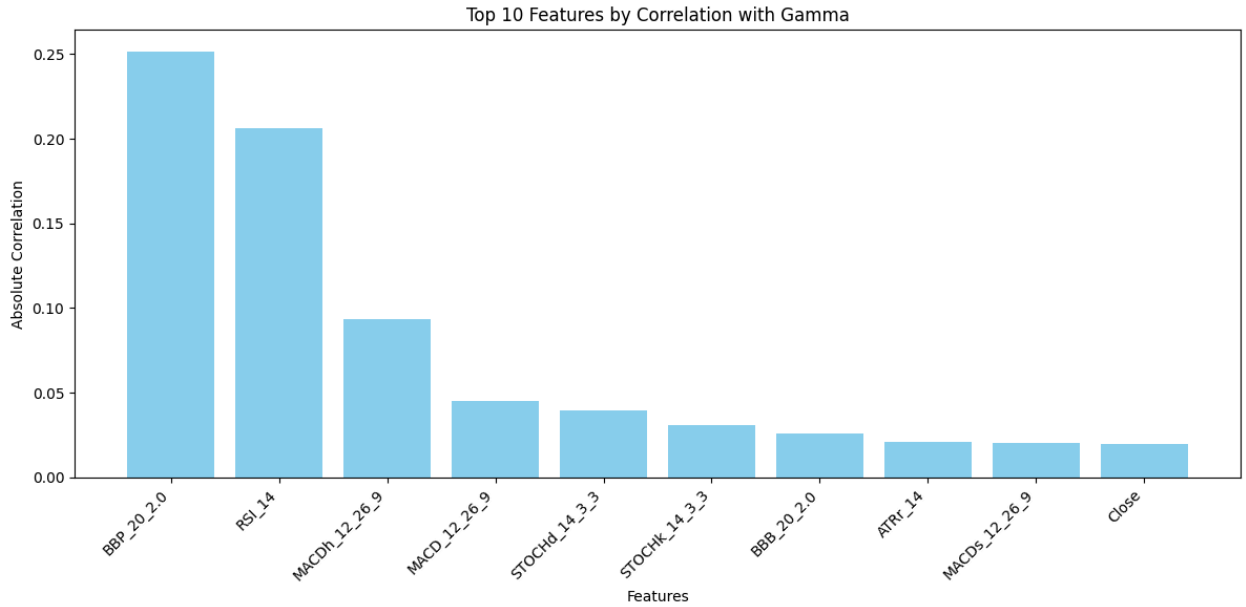
'MACDs_12_26_9', 'Close'],

dtype='object')

Selected features saved to ECH_selected_features_correlation.csv

Feature Correlation with Gamma

0	Gamma	1.000000
1	BBP_20_2.0	0.251631
2	RSI_14	0.206282
3	MACDh_12_26_9	0.093332
4	MACD_12_26_9	0.045117
5	STOCHd_14_3_3	0.039653
6	STOCHk_14_3_3	0.031081
7	BBB_20_2.0	0.025680
8	ATRr_14	0.020945
9	MACDs_12_26_9	0.020146



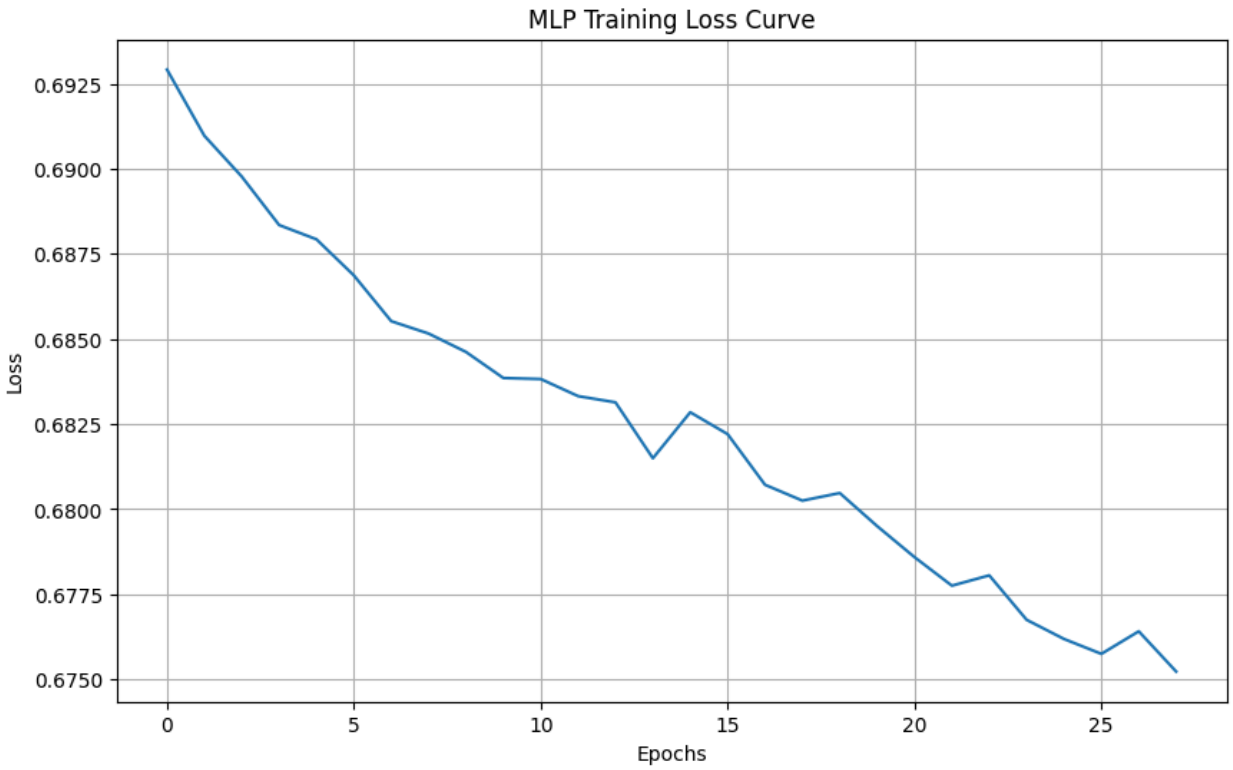
Model Accuracy: 56.92%
Classification Report:

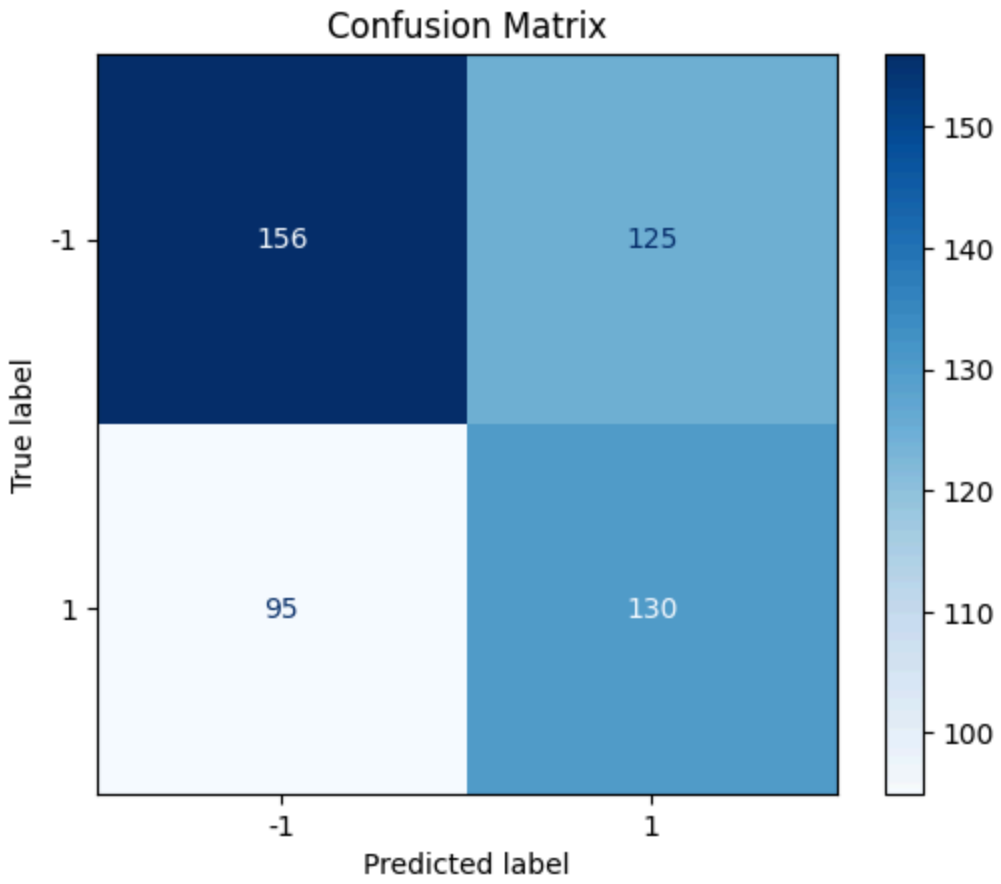
	precision	recall	f1-score	support
-1	0.67	0.43	0.53	281
1	0.51	0.74	0.60	225
accuracy			0.57	506
macro avg	0.59	0.59	0.57	506
weighted avg	0.60	0.57	0.56	506

Model Accuracy: 62.06%

Classification Report as Table:

	precision	recall	f1-score	support
-1	0.706977	0.540925	0.612903	281.000000
1	0.556701	0.720000	0.627907	225.000000
accuracy	0.620553	0.620553	0.620553	0.620553
macro avg	0.631839	0.630463	0.620405	506.000000
weighted avg	0.640155	0.620553	0.619575	506.000000


MLP Training Loss Curve
Loss
Epochs
0.6925
0.6900
0.6875
0.6850
0.6825
0.6800
0.6775
0.6750
0
5
10
15
20
25



Step 3(d)

BBP_20_2.0 19

RSI_14 14

MACDh_12_26_9 33

MACD_12_26_9 25

STOCHd_14_3_3 17

STOCHk_14_3_3 15

BBB_20_2.0 19

ATRr_14 14

MACDs_12_26_9 33

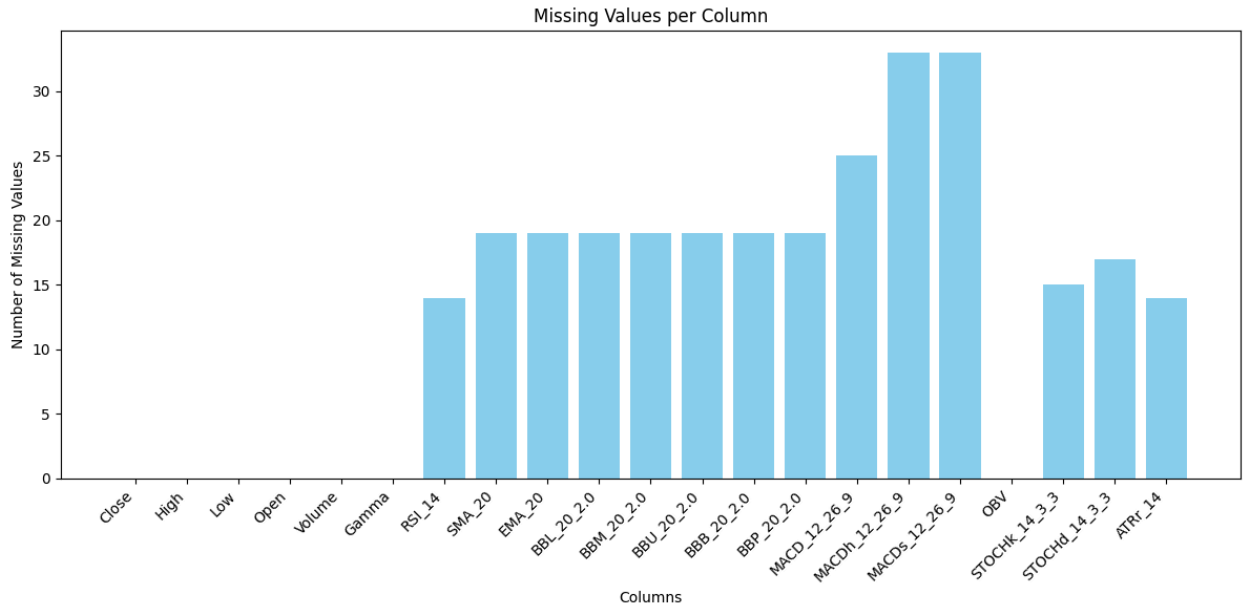
Close 0

Gamma 0

dtype: int64

Missing Values Table:

	Missing Values
Close	0
High	0
Low	0
Open	0
Volume	0
Gamma	0
RSI_14	14
SMA_20	19
EMA_20	19
BBL_20_2.0	19
BBM_20_2.0	19
BBU_20_2.0	19
BBB_20_2.0	19
BBP_20_2.0	19
MACD_12_26_9	25
MACDh_12_26_9	33
MACDs_12_26_9	33
OBV	0
STOCHk_14_3_3	15
STOCHd_14_3_3	17
ATRr_14	14


Missing Values per Column
Number of Missing Values
0
5
10
15
20
25
30
Close
High
Low
Open
Volume
Gamma
RSI_14
SMA_20
EMA_20
BBL_20_2.0
BBM_20_2.0
BBU_20_2.0
BBB_20_2.0
BBP_20_2.0
MACD_12_26_9
MACDh_12_26_9
MACDs_12_26_9
OBV
STOCHk_14_3_3
STOCHd_14_3_3
ATRr_14
Columns

10-Fold Cross-Validation Accuracy: 58.01%

Accuracy for Each Fold:

Fold 1: 63.64%

Fold 2: 59.29%

Fold 3: 54.55%

Fold 4: 62.45%

Fold 5: 58.89%

Fold 6: 57.31%

Fold 7: 49.80%

Fold 8: 60.47%

Fold 9: 53.75%

Fold 10: 59.92%

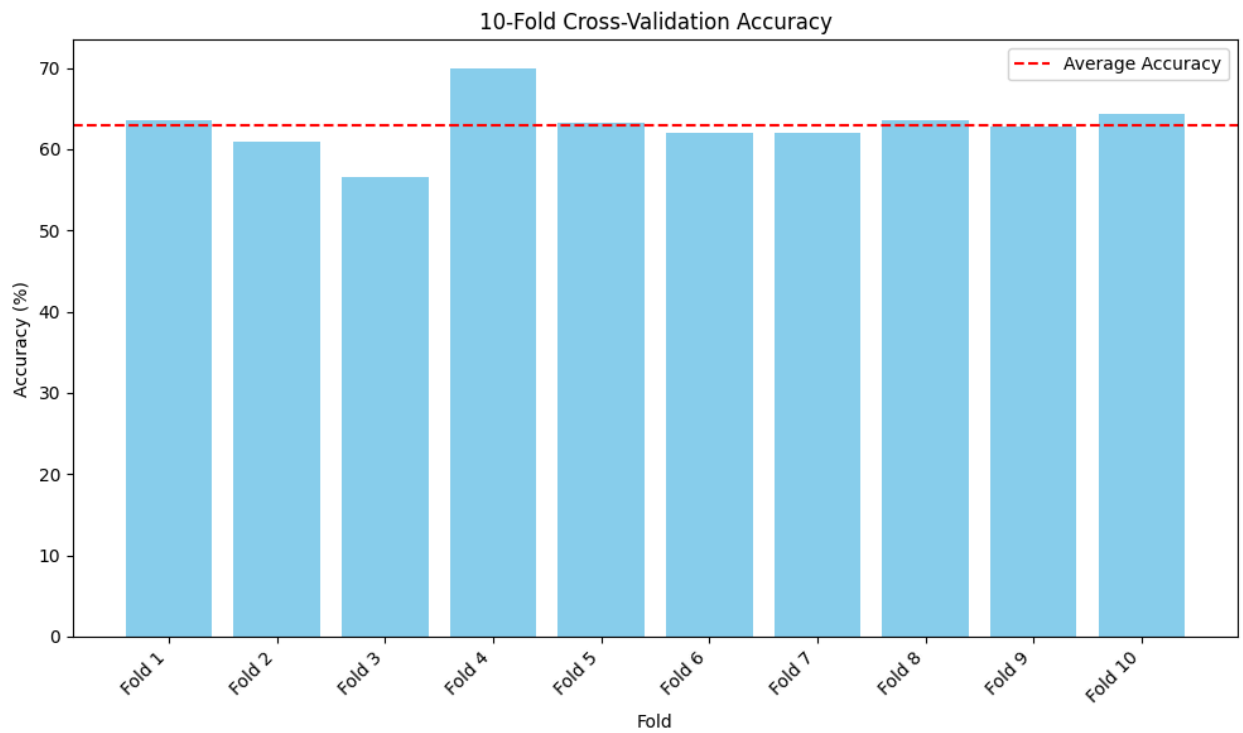
10-Fold Cross-Validation Results:

	Fold	Accuracy (%)
0	Fold 1	63.636364
1	Fold 2	60.869565
2	Fold 3	56.521739
3	Fold 4	69.960474
4	Fold 5	63.241107
5	Fold 6	62.055336
6	Fold 7	62.055336
7	Fold 8	63.636364
8	Fold 9	62.845850
9	Fold 10	64.285714
10	Average	62.910785

10-Fold Cross-Validation Results:

	Fold	Accuracy (%)
0	Fold 1	63.636364

1	Fold 2	60.869565
2	Fold 3	56.521739
3	Fold 4	69.960474
4	Fold 5	63.241107
5	Fold 6	62.055336
6	Fold 7	62.055336
7	Fold 8	63.636364
8	Fold 9	62.845850
9	Fold 10	64.285714
10	Average	62.910785



Final model saved to final_mlp_model.pkl

Final model saved to final_mlp_model.pkl

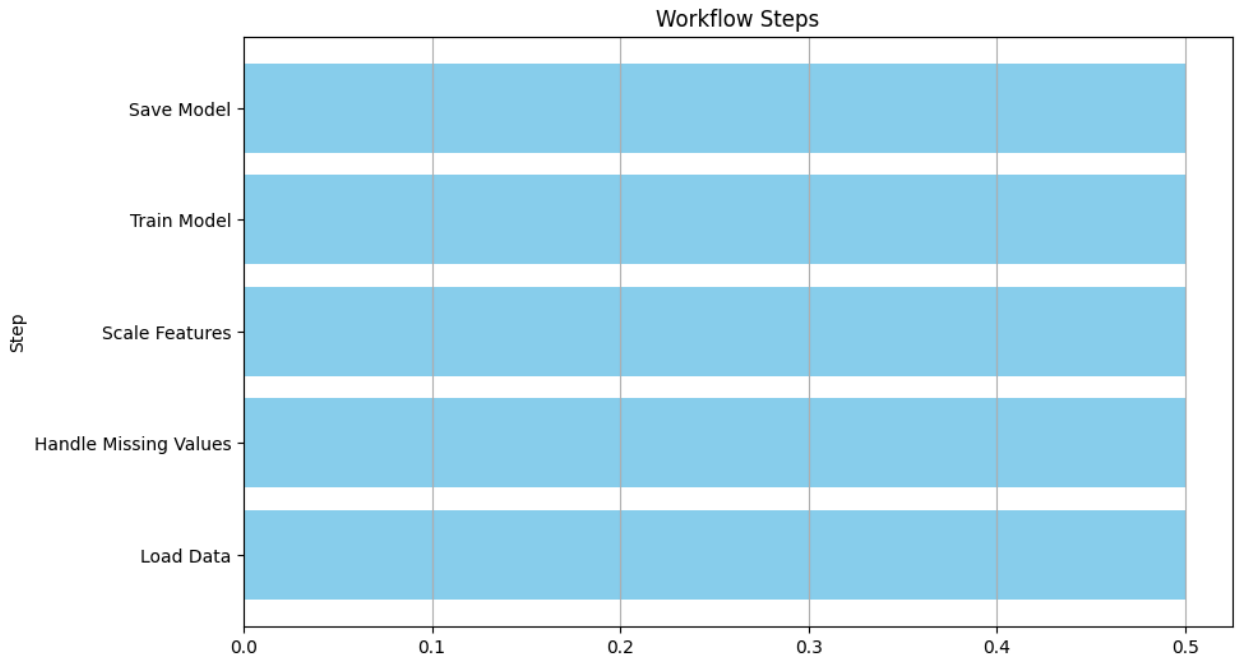
Summary of Workflow:

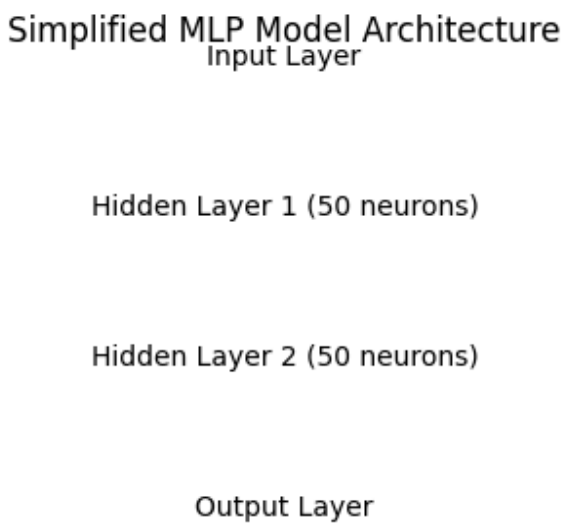
	Step	Description
0	**Load Data**	**Loaded dataset from ECH_selected_features_corr...**
1	**Handle Missing Values**	**Imputed missing values using the mean strategy**
2	**Scale Features**	**Scaled features using StandardScaler**
3	**Train Model**	**Trained MLPClassifier with two hidden layers o...**
4	**Save Model**	**Saved model as final_mlp_model.pkl**

Final model saved to final_mlp_model.pkl

Summary of Workflow:

	Step	Description
0	Load Data	Loaded dataset from ECH_selected_features_corr...
1	Handle Missing Values	Imputed missing values using the mean strategy
2	Scale Features	Scaled features using StandardScaler
3	Train Model	Trained MLPClassifier with two hidden layers o...
4	Save Model	Saved model as final_mlp_model.pkl





Final model loaded from final_mlp_model.pkl
Test Set Accuracy: 47.04%
Test Set Classification Report:

	precision	recall	f1-score	support
-1	0.64	0.10	0.18	281
1	0.45	0.93	0.61	225

```
    accuracy                    0.47      506
   macro avg      0.55      0.52      0.39      506
weighted avg      0.56      0.47      0.37      506
```

10-Fold Cross-Validation Accuracy: 58.01%

Accuracy for Each Fold:
Fold 1: 63.64%
Fold 2: 59.29%
Fold 3: 54.55%
Fold 4: 62.45%
Fold 5: 58.89%
Fold 6: 57.31%
Fold 7: 49.80%
Fold 8: 60.47%
Fold 9: 53.75%
Fold 10: 59.92%

Final model loaded from final_mlp_model.pkl
Test Set Accuracy: 47.04%

Performance Summary:

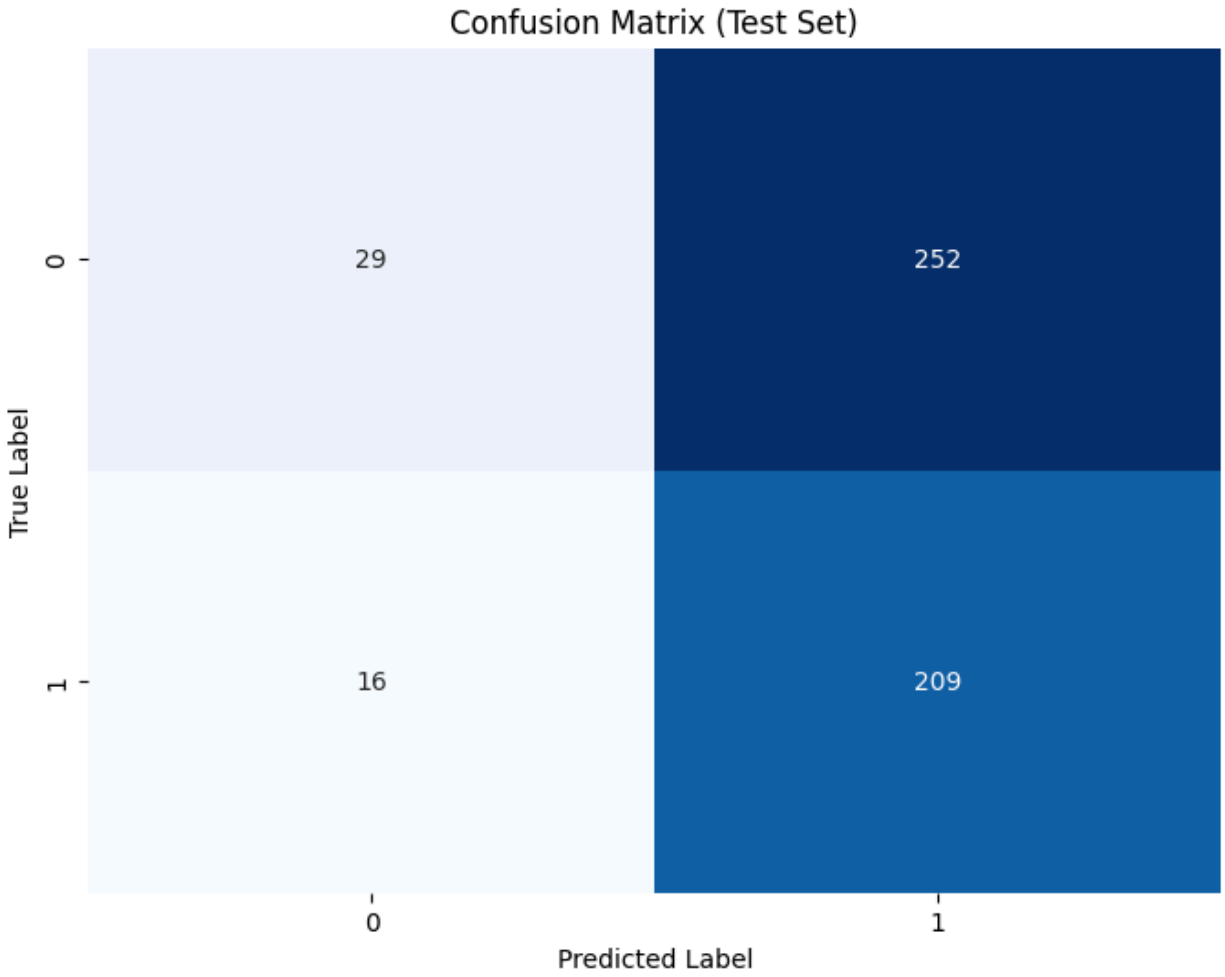
	Metric	Value (%)
0	Test Set Accuracy	47.035573
1	Fold 1 Accuracy	63.636364
2	Fold 2 Accuracy	59.288538
3	Fold 3 Accuracy	54.545455
4	Fold 4 Accuracy	62.450593
5	Fold 5 Accuracy	58.893281
6	Fold 6 Accuracy	57.312253
7	Fold 7 Accuracy	49.802372
8	Fold 8 Accuracy	60.474308
9	Fold 9 Accuracy	53.754941
10	Fold 10 Accuracy	59.920635
11	Average CV Accuracy	58.007874

Final model loaded from final_mlp_model.pkl

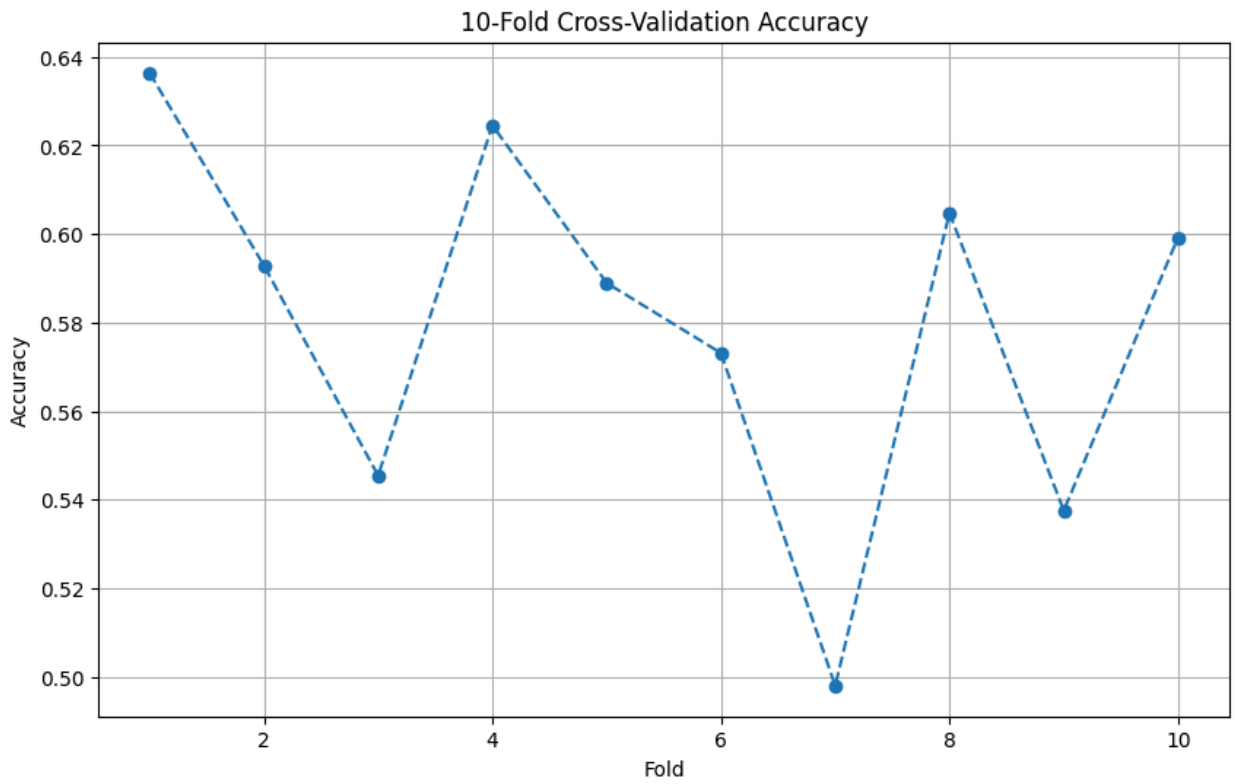
Test Set Accuracy: 47.04%

Test Set Classification Report:

	precision	recall	f1-score	support
-1	0.64	0.10	0.18	281
1	0.45	0.93	0.61	225
accuracy			0.47	506
macro avg	0.55	0.52	0.39	506
weighted avg	0.56	0.47	0.37	506



10-Fold Cross-Validation Accuracy: 58.01%


10-Fold Cross-Validation Accuracy
Accuracy
Fold
0.64
0.62
0.60
0.58
0.56
0.54
0.52
0.50
2
4
6
8
10

References:

Almehmadi, A. (2021). Covid-19 pandemic data predict the stock market. Computer Systems Science & Engineering, 36(3), 451–460.

Ananthi, M., & Vijayakumar, K. (2021). Stock market analysis using candlestick regression and market trend prediction (CKRM). Journal of Ambient Intelligence and Humanized Computing, 12(5), 4819–4826.

Antoniewicz, R. S., & Heinrichs, J. (2014). Understanding exchange-traded funds: How ETFS work. Jane, Understanding Exchange-Traded Funds: How ETFs Work (September 30, 2014).

Barra, S., Carta, S. M., Corriga, A., Podda, A. S., & Recupero, D. R. (2020). Deep learning and time series-to-image encoding for financial forecasting. IEEE/CAA Journal of Automatica Sinica, 7(3), 683–692.

Cagliero, L., Garza, P., Attanasio, G., & Baralis, E. (2020). Training ensembles of faceted classification models for quantitative stock trading. Computing, 102, 1213–1225.

Chen, M.-Y., & Chen, B.-T. (2015). A hybrid fuzzy time series model based on granular computing for stock price forecasting. Information Sciences, 294, 227–241.

Chen, M.-Y., Liao, C.-H., & Hsieh, R.-P. (2019). Modeling public mood and emotion: Stock market trend prediction with anticipatory computing approach. Computers in Human Behavior, 101, 402–408.

Chen, W., Jiang, M., Zhang, W.-G., & Chen, Z. (2021a). A novel graph convolutional feature based convolutional neural network for stock trend prediction. Information Sciences, 556, 67–94.

Chen, W., Jiang, M., Zhang, W.-G., & Chen, Z. (2021b). A novel graph convolutional feature based convolutional neural network for stock trend prediction. Information Sciences, 556, 67–94.

De Haan, L., Mercadier, C., & Zhou, C. (2016). Adapting extreme value statistics to financial time series: Dealing with bias and serial dependence. Finance and Stochastics, 20(2), 321–354.

Deville, L. (2008). Exchange traded funds: History, trading, and research. Handbook of Financial Engineering (pp. 67–98). Springer.

Ecer, F., Ardabili, S., Band, S. S., & Mosavi, A. (2020). Training multilayer perceptron with genetic algorithms and particle swarm optimization

for modeling stock price index prediction. Entropy, 22(11), 1239.

Ehlers, J. F. (2013). Cycle analytics for traders,+ downloadable software: Advanced technical trading concepts. John Wiley & Sons.

Fang, W., Zhang, S., & Xu, C. (2024). Improving prediction efficiency of Chinese stock index futures intraday price by VIX-Lasso-GRU Model. Expert Systems with Applications, 238, 121968.

Ferreira, A. J., & Figueiredo, M. A. (2012). Efficient feature selection filters for high-dimensional data. Pattern Recognition Letters, 33(13), 1794–1804.

Hegde, S. P., & McDermott, J. B. (2004). The market liquidity of diamonds, q's, and their underlying stocks. Journal of Banking & Finance, 28(5), 1043–1067.

Jamshed, H., Khan, M., Khurram, M., Inayatullah, S., & Athar, S. (2019). Data preprocessing: A preliminary step for web data mining. 3c Tecnología: Glosas de innovación aplicadas a la pyme, 8(29), 206–221.

Jiang, M., Jia, L., Chen, Z., & Chen, W. (2020). The two-stage machine learning ensemble models for stock price prediction by combining mode decomposition, extreme learning machine and improved harmony search algorithm. Annals of Operations Research, 309(2), 1–33.

Johnson, K. (2021). Pandas - technical analysis. https://github.com/twop llc/pandas-ta.

Liang, Q., Rong, W., Zhang, J., Liu, J., & Xiong, Z. (2017). Restricted Boltzmann machine based stock market trend prediction. In: 2017 International Joint Conference on Neural Networks (IJCNN) (pp. 1380–1387). IEEE.

Lu, Y., Cohen, I., Zhou, X. S., & Tian, Q. (2007). Feature selection using principal feature analysis. In Proceedings of the 15th ACM International Conference on Multimedia, MM '07 (pp. 301–304). New York, NY, USA: Association for Computing Machinery.

Matuozzo, A., Yoo, P. D., & Provetti, A. (2023). A right kind of wrong: European equity market forecasting with custom feature engineering and loss functions. Expert Systems with Applications, 223,

119854.

Miralles-Quirós, J. L., Miralles-Quirós, M. M., & Nogueira, J. M. (2019). Diversification benefits of using exchange-traded funds in compliance to the sustainable development goals. Business Strategy and the Environment, 28(1), 244–255.

Nelson, D. M., Pereira, A. C., & de Oliveira, R. A. (2017). Stock market's price movement prediction with LSTM neural networks. In 2017 International Joint Conference on Neural Networks (IJCNN) (pp. 1419–1426). IEEE.

O'Hara, H. T., Lazdowski, C., Moldovean, C., & Samuelson, S. T. (2000). Financial indicators of stock price performance. American Business Review, 18(1), 90.

Reddy, G. T., Reddy, M. P. K., Lakshmanna, K., Kaluri, R., Rajput, D. S., Srivastava, G., & Baker, T. (2020). Analysis of dimensionality reduction techniques on big data. IEEE Access, 8, 54776–54788.

Shearer, C. (2000). The CRISP-DM model: The new blueprint for data mining. Journal of Data Warehousing, 5(4), 13–22.

Spearman, C. (1961). The proof and measurement of association between two things. Appleton-Century-Crofts.

Sun, W., Cai, Z., Liu, F., Fang, S., & Wang, G. (2017). A survey of data mining technology on electronic medical records. In: 2017 IEEE 19th International Conference on e-Health Networking, Applications and Services (Healthcom) (pp. 1–6).

Tang, H., Dong, P., & Shi, Y. (2019). A new approach of integrating piecewise linear representation and weighted support vector machine for forecasting stock turning points. Applied Soft Computing, 78, 685–696.

Verma, S., Sahu, S. P., & Sahu, T. P. (2022). Stock market analysis of beauty industry during covid-19. In: Advances in Data Computing, Communication and Security: Proceedings of I3CS2021 (pp. 157–168), Springer.

Verma, S., Sahu, S. P., & Sahu, T. P. (2023a). Portfolio management using additive ratio assessment based stock selection and deep learning for prediction. International Journal of Information Technology, 15(8), 4055–4062.

Verma, S., Sahu, S. P., & Sahu, T. P. (2023b). Stock market forecasting with different input indicators using machine learning and deep learning techniques: A review. Engineering Letters, 31(1), 213–229.

Verma, S., Sahu, S. P., & Sahu, T. P. (2023c). Two-stage hybrid feature selection approach using levy's flight based chicken swarm optimization for stock market forecasting. Computational Economics, 1–32. https://doi.org/10.1007/s10614-023-10400-8.

Zhang, J., Li, L., & Chen, W. (2021). Predicting stock price using two-stage machine learning techniques. Computational Economics, 57(4), 1237–1261.

Zhao, J., Zeng, D., Liang, S., Kang, H., & Liu, Q. (2021). Prediction model for stock price trend based on recurrent neural networks. Journal of Ambient Intelligence and Humanized Computing, 12, 745–753.